Scored Consistency Distillation of SD3.5-Medium: Selecting Teacher Trajectories with a Reference Photograph

Technical report

September 2026

Abstract

We distil Stable Diffusion 3.5 Medium into a 4-step student with consistency distillation. For every training caption the frozen teacher generates four candidate trajectories from four seeds. A scorer picks one of the four, and the student is trained on that trajectory only. The baseline picks at random. The main scorer is the cosine similarity between DINOv2 mean-pooled patch features of the candidate image and of the real photograph that the caption describes. It uses no text model. On 3,000 COCO captions, three seeds, and 6,000 updates, this selection improves T2I-CompBench by +0.014 and GenEval2 by +2.2 points over random selection, with 95% bootstrap intervals that exclude zero, and does not cost fidelity. At 118k captions and 57k updates, three training seeds, with weight averaging of the last five checkpoints, the gain is +0.0175 CompBench ($p = 10^{-14}$) and +1.3 GenEval2; it is +0.013 under the official ten-images-per-prompt CompBench protocol ($p = 10^{-41}$), and the scored student has better CMMD, precision and recall on every seed. The gain is unchanged by the optimizer batch size, does not grow with eight candidates instead of four, and vanishes with a stronger teacher. Argmax is the selection rule to use: exact Boltzmann weighting of all four candidates shows no detected improvement over it at about three times the wall-clock, softer weightings are worse, and drawing one candidate per visit from the same weights matches exact weighting once checkpoints are averaged. A 6.8M-parameter projector that scores terminal latents against the photograph without decoding recovers about three quarters of the selection gain; used as a reward inside training it is optimised by the student without the true score following, and is not adopted. The exact version of that reward, the DINOv2 score of the decoded prediction with gradients through the decoder, does work. In a five-seed factorial with the caption order matched across arms it adds +0.0055 CompBench ($p = 0.004$) over argmax selection, is additive with selection, which stays worth +0.010 with the reward present, lowers CMMD from 0.79 to 0.73 on every seed, and raises the held-out similarity of the student's own 4-step samples to the photographs; selection with the reward is +0.017 CompBench and +1.9 GenEval2 over naive distillation, for 17% more training time and no change at inference. A one-factor ablation shows the reward under-weighted at that setting: doubling and quadrupling its weight, or rewarding all five supervised states, adds a further +0.006 to +0.011 CompBench and lowers CMMD to 0.63 to 0.69, while the resize and precision of the differentiable path do not matter and the projector surrogate stays null however hard it is refreshed. Combining the heavier weight with all five states adds a further +0.0153 CompBench over the recipe (three seeds, $p = 0.012$) and matches the heaviest single-factor weight's CMMD without its GenEval2 cost, but past what either factor gives alone the gain trades off against GenEval2 and FID rather than adding cleanly; the same reward on two stronger teachers moves CompBench in the same direction as on the 8-step teacher but is not statistically resolved at three seeds. At the full 118k scale the reward (unmodified, $\lambda = 15.504$) adds +0.0087 CompBench and +1.45 GenEval2 over argmax selection alone, every seed positive, and lowers CMMD by a further 0.10 on top of selection's own gain, confirming the 3k-scale result at 40 times the caption count. The same pipeline with VQAScore as the scorer gives similar gains but shares a model family with the evaluators. This document gives the full recipe: data, model, scorers, loss, hyperparameters, evaluation protocol, results, ablations and qualitative examples.

Contents

1 Summary

Method in one paragraph. The teacher is SD3.5-Medium with classifier-free guidance $w = 7$ and 8 Euler steps. For each caption we roll out $N = 4$ candidates from four fixed seeds. A scorer ranks the four decoded images and the best one is stored in a manifest. Training re-rolls only the chosen trajectory and regresses the student's clean-image estimate at a noisier state onto the teacher's clean-image estimate at a less noisy state. The student is sampled with 4 steps and no guidance. The only thing that differs between arms is which one of the four trajectories the student sees.

Where things are defined. Section 7 states the single objective every trained model minimises, tabulates every arm as a choice of weights, estimator, score and reward (Table 6), lists every network with its size (Table 7), and compares all 3k-pool arms in four figures. Two names recur: the *RGB scorer* is the DINOv2 score of the decoded candidate image, and the *latent scorer* is the projector that scores the candidate's latent without decoding it. The projector is used in three settings, in this order: frozen as an offline selector, frozen as a reward inside training, and

refreshed during training (Table 8); a fourth setting replaces it by the RGB scorer itself, decoded inside the loop, and is the reference the projector rewards were approximating (Section 11.7).

Main numbers. Table 1 gives the headline results. All deltas are paired against the random-selection student on identical prompts.

Table 1: Headline results. Alignment deltas are against the random-selection student. The 3k rows use three training seeds; intervals are 95% hierarchical bootstrap over seeds and prompts. The averaged 3k rows (Section 11 protocol: average of the 2k/4k/6k checkpoints; the factorial of Section 11.8, caption order matched across arms) give seed-paired mean ± sd over seeds with the seed t-test p. The 118k rows use three training seeds and weight-averaged checkpoints, with per-prompt paired tests pooled over seeds; the VQAScore arm at 118k is one seed. CMMD is the seed mean.

Setting	Scorer	CompBench Δ	GenEval2 Δ (×100)	CMMD ↓
3k captions, 6k updates	random (baseline)	0.4584	20.73	0.963
3k captions, 6k updates	DINOv2 patches	+0.0140 [+0.0051, +0.0237]	+2.17 [+0.24, +4.08]	0.893
3k captions, 6k updates	VQAScore	+0.0233 [+0.0097, +0.0364]	+2.34 [+0.55, +4.05]	0.883
3k, averaged checkpoints, 5 seeds	random (baseline)	0.4751	22.47	0.85
3k, averaged checkpoints, 5 seeds	DINOv2 patches	$+0.0117 \pm 0.0058$ ($p = 0.011$)	$+1.46 \pm 1.38$	0.79
3k, averaged checkpoints, 5 seeds	DINOv2 patches + exact DINO reward	$+0.0172 \pm 0.0051$ ($p = 0.002$)	$+1.94 \pm 0.68$ ($p = 0.003$)	0.73
118k captions, 57k updates, 3 seeds	random (baseline)	0.4668	20.29	0.84
118k captions, 57k updates, 3 seeds	DINOv2 patches	+0.0175 ($p = 10^{-14}$)	+1.29 ($p = 0.02$)	0.78
118k, official 10-image CompBench, 3 seeds	DINOv2 patches	+0.0131 ($p = 2 \times 10^{-41}$)	–	–
118k captions, 57k updates, 1 seed	VQAScore	+0.0185 ($p = 10^{-7}$)	+3.05 ($p = 0.001$)	0.79
Teacher, 28 steps, $w = 7$	–	0.5053	17.05	0.64

2 Base model

2.1 SD3.5-Medium as a rectified flow

SD3.5-Medium is a 2.5B-parameter Multimodal Diffusion Transformer (MMDiT). We use it unchanged. It generates in the latent space of its VAE. At 512 × 512 pixels a latent is $z \in \mathbb{R}^{16\times64\times64}$, so $D = 65{,}536$.

Rectified flow defines a straight path between a clean latent x_0 and Gaussian noise $\varepsilon \sim \mathcal{N}(0, I)$:

$$z_\sigma = (1 - \sigma)\, x_0 + \sigma\, \varepsilon, \qquad \sigma \in [0, 1]. \tag{1}$$

The network predicts the velocity $v = \varepsilon - x_0$. Sampling is Euler integration down a σ grid:

$$z_{k+1} = z_k + (\sigma_{k+1} - \sigma_k)\, v_\theta(z_k, \sigma_k, c). \tag{2}$$

The network is conditioned on the timestep $t = 1000\,\sigma$. The grid is built by the scheduler with a shift $\lambda = 3$: $\sigma \mapsto \lambda\sigma/(1 + (\lambda - 1)\sigma)$. The grids used in this work are given in Table 2.

Table 2: σ grids. The 4-step grid is not a subset of the 8-step grid; only the endpoints coincide. It is a subset of the 28-step grid (every ninth state).

Grid	σ values
8 steps (teacher, training)	1, 0.948, 0.883, 0.801, 0.694, 0.548, 0.338, 0.009, 0
4 steps (student, inference)	1, 0.858, 0.602, 0.009, 0
28 steps (teacher reference)	1 down to 0, same shift, 28 intervals

2.2 Conditioning

The caption enters through three frozen text encoders: CLIP-L and CLIP-G as token sequences and a pooled 2048-d vector, and T5-XXL as a token sequence. The teacher uses classifier-free guidance:

$$v^{\text{cfg}} = v_\theta(z, \sigma, \varnothing) + w\big(v_\theta(z, \sigma, c) - v_\theta(z, \sigma, \varnothing)\big), \qquad w = 7. \tag{3}$$

The VAE is frozen and is used to decode candidates for scoring and to decode images for evaluation; in the exact-reward arm of Section 11.7 it also decodes the student's predictions inside the training loop, with gradients passing through it.

3 Data

3.1 Training captions and reference photographs

Training captions come from COCO `train2017`. Each caption is paired with the photograph it was written for. A caption string that COCO attaches to more than one photograph is kept only for its first photograph, so every caption maps to exactly one reference. Two pools are used:

- **3k pool**: 3,000 captions drawn at random with a fixed seed. Used for every three-seed experiment and every ablation.
- **118k pool**: 113,948 captions (named after its initial count of 118,287 before deduplication and reference checks). Used for the single-seed scale experiment.

The reference photograph is only used by the scorer. It never enters training.

3.2 Evaluation pools

All evaluation uses one sealed pool built once with a fixed seed (Table 3). Evaluation references and fidelity captions come from COCO `val2017`, so no training caption or photograph appears in any evaluation set.

Table 3: Evaluation pools. Every arm and every seed is evaluated on exactly these prompts.

Pool	Prompts	Source
T2I-CompBench	2,398	300 per category (298 for 2D-spatial), 8 categories, official prompt files
GenEval2	800	official GenEval2 prompts, 3 to 10 atoms each, 100 per atom count
COCO VQAScore (in-domain)	5,000	`val2017` captions, one per image
Fidelity	5,000	`val2017` captions; reals are the 5,000 matching photographs

4 Candidate pool

For each caption we draw $N = 4$ candidates. Candidate j of caption `idx` uses the seed $1000 \cdot \texttt{idx} + j$, so the whole pool is reproducible from integers. Each candidate is an 8-step Euler rollout of the teacher at $w = 7$ and 512×512:

$$\varepsilon^{(j)} \sim \mathcal{N}(0, I) \text{ seeded by } 1000 \cdot \texttt{idx} + j, \qquad \hat{x}_0^{(j)} = \text{Euler}_{K=8}\big(\varepsilon^{(j)};\, v^{\text{cfg}}, c\big). \tag{4}$$

Each candidate is decoded to an image $I_j = \text{Dec}(\hat{x}_0^{(j)})$ for scoring. The four candidates differ only by their noise seed. The training rollout in Section 6 uses the same sampler and the same seed, so the cached candidate and the training trajectory are the same object.

Nothing is stored except the seed, the scores and the chosen index. Trajectories are re-rolled from the seed during training.

5 Scorers

A scorer assigns a number S_j to each candidate. Selection is hard:

$$j^\star = \arg\max_{j \in \{1,\dots,4\}} S_j. \tag{5}$$

The index $j^\star$ is computed once, offline, and stored in a manifest keyed by caption. No gradient ever flows through a scorer. Section 11 compares this rule with Boltzmann-weighted and sampled selection; none of them is detectably better than argmax. The random baseline replaces $j^\star$ by a uniform draw that is also stored in the manifest and is therefore identical across training seeds.

5.1 DINOv2 mean-pooled patches (main scorer)

We use DINOv2-B (`facebook/dinov2-base`, $d = 768$) with its standard preprocessing: resize the short side to 256, centre-crop to 224×224, ImageNet normalisation. The model returns one CLS token and $P = 256$ patch tokens $h_1, \dots, h_P$ (a 16×16 grid, patch size 14). We drop the CLS token, average the patch tokens and normalise:

$$u^{\mathrm{pat}}(I) = \frac{\frac{1}{P}\sum_{p=1}^{P} h_p}{\left\|\frac{1}{P}\sum_{p=1}^{P} h_p\right\|}. \tag{6}$$

The score is the cosine similarity to the reference photograph x^{ref}:

$$S_j = \left\langle u^{\mathrm{pat}}(I_j),\, u^{\mathrm{pat}}(x^{\mathrm{ref}}) \right\rangle. \tag{7}$$

Because this scorer reads the decoded image, it is called the *RGB scorer* in the rest of this document, in contrast to the *latent scorer* of Section 11.5, which reads the candidate's latent and never decodes it. Candidates are decoded at 512×512. The reference photograph is first resized (squashed, aspect ratio not preserved) to 512×512 with bicubic interpolation, so both enter the DINO processor as squares and the centre crop discards nothing. There is no text model in this scorer. The caption enters only through its photograph.

5.2 Other scorers

- **DINOv2 CLS.** Same model and preprocessing, but the normalised CLS token $h_0/\|h_0\|$ instead of the patch mean.
- **CLIP-I.** Cosine between CLIP ViT-L/14 image embeddings of the candidate and of the reference photograph.
- **VQAScore.** The probability that CLIP-FlanT5-XXL (11B) answers "yes" to "Does this figure show *caption*?". It reads the caption and needs no photograph. It shares a model family with the BLIP-VQA part of CompBench and the VLM judge of GenEval2, so its gains are partly circular. It is the reference scorer, not the proposed one.
- **ImageReward.** A BLIP-based learned human-preference model that reads the caption.
- **PickScore.** A CLIP-H model fine-tuned on human preferences that reads the caption.

5.3 How well each scorer selects

Table 4 measures selection quality offline on the 3k pool. The ground truth is VQAScore. The random pick scores 0.8429 on average and the best-of-four oracle 0.9334, so the headroom is 0.0905. The table gives the fraction of that headroom each scorer recovers and how often its pick is the oracle pick (chance is 25%).

Table 4: Offline selection quality on the 3k pool, 12,000 candidates. Headroom is measured against the VQAScore best-of-four oracle, so the caption-reading scorers have an advantage here that is partly shared with the ground truth.

Scorer	Needs	VQAScore headroom recovered	Top-1 agrees with oracle
VQAScore (oracle)	caption, 11B VLM	100%	1.000
ImageReward	caption, BLIP	64.5%	0.427
PickScore	caption, CLIP-H	55.6%	0.415
DINOv2 patches	reference photo	43.6%	0.356
CLIP-I	reference photo	41.4%	0.328
DINOv2 CLS	reference photo	39.9%	0.324

The patch scorer and the CLS scorer agree on only 54.5% of picks, so they are different selectors. ImageReward and PickScore agree with each other on 50.0% of picks and with VQAScore on 42.7% and 41.5%.

6 Distillation

6.1 Teacher and student

The teacher is the pretrained SD3.5-Medium transformer with frozen weights θ_T, kept in bf16 with no gradient, always evaluated with guidance $w = 7$. It is never updated. There is no exponential moving average, no momentum target and no stop-gradient copy of the student. The student v_θ is a full copy of the same transformer, initialised from θ_T, and is the only network that trains. Every target is a deterministic function of the frozen teacher's own trajectory, so it does not depend on θ and is identical across epochs.

6.2 Trajectory and targets

For each training caption we roll out only the selected candidate with the teacher, $w = 7$, $K = 8$ steps, giving $z_0, z_1, \ldots, z_8$. The supervised states are the scoring window $[0.4, 0.9]$ of the grid, $\mathcal{K} = \{3, 4, 5, 6, 7\}$, so $K_w = 5$ states per update.

For each $k \in \mathcal{K}$ the teacher's realised chord across its own step is

$$\bar{v}_k = \frac{z_{k+1} - z_k}{\sigma_{k+1} - \sigma_k}, \tag{8}$$

and the clean-latent estimate implied by that chord is

$$\tilde{x}_0^{(k)} = z_k - \sigma_k \, \bar{v}_k. \tag{9}$$

This is exact algebra for a straight path: substituting $z_k = (1 - \sigma_k)x_0 + \sigma_k \varepsilon$ and $\bar{v} = \varepsilon - x_0$ recovers x_0.

6.3 Student input and loss

The student is placed at a noisier state. For each k we draw $\Delta_k \sim \mathcal{U}\{1, 2, 3\}$, independently per state and afresh on every visit, and set $s = k - \Delta_k$. The five inputs $z_{k-\Delta_k}$ are batched into one forward pass. The student uses one conditional pass, no guidance, and forms its own clean-latent estimate:

$$\hat{x}_0^{(s)} = z_s - \sigma_s \, v_\theta(z_s, \sigma_s, c). \tag{10}$$

The loss is a pseudo-Huber distance in x_0 space, averaged over the window:

$$\mathcal{L}(\theta) = \frac{1}{K_w} \sum_{k \in \mathcal{K}} \left[\sqrt{\left\| \hat{x}_0^{(k-\Delta_k)} - \mathrm{sg}\left[\tilde{x}_0^{(k)}\right] \right\|_2^2 + c^2} - c \right], \qquad c = 0.00054\sqrt{D} \approx 0.138. \tag{11}$$

The norm is summed over all D latent dimensions, so this is one distance per sample. sg[·] is stop-gradient. The student stands at a noisier point and must name the clean image that the teacher only reaches from a less noisy point. Composing these skips is what turns 8 teacher steps into 4 student steps.

Because the student trains with a single conditional pass against $w = 7$ teacher trajectories, guidance is absorbed into its conditional prediction. The student must be sampled at $w = 1$. Sampling it at $w = 7$ applies guidance twice and roughly halves its scores.

6.4 Training details

Table 5 lists both training configurations. Each optimiser step processes one caption per GPU: one teacher rollout of the selected candidate (16 teacher forward passes, 8 steps with a conditional and an unconditional branch) and the five batched student passes. The training seed controls only the Δ_k draws and the caption order. The selection index is fixed in the manifest.

Table 5: Training configurations. Both use the same frozen trainer code.

	3k configuration	118k configuration
Captions	3,000	113,948
GPUs (H200)	1	4, DDP, one caption per GPU per step
Updates	6,000 (2 passes)	56,974 (2 passes)
Seeds	3	3 (VQAScore arm: 1)
Optimiser	AdamW, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$	same
Weight decay	0	0
Learning rate	10^{-5}	2×10^{-5}
Warm-up	300 steps, linear, then constant	1,000 steps, linear, then constant
Gradient clip	global norm 1.0	global norm 1.0
Precision	fp32 master weights, bf16 autocast	same
Gradient checkpointing	on	on
Teacher, text encoders, VAE	frozen, bf16	same
Captions per update	1	4 (one per GPU; 16 with accumulation, Section 10.5)
Reported weights	final step	uniform average of steps 40k, 45k, 50k, 55k, final
Wall-clock per run	1.6 h	12.5 h
Peak GPU memory	–	66 GB per GPU

The raw gradient norm of this loss is of order 10^2 throughout training, so the clip is active on every step. Every update is therefore a normalised-gradient step of size η. All arms share this, so it does not affect comparisons, but the learning rate and the clip threshold are not separately tunable.

6.5 Weight averaging at 118k

The 118k runs save a checkpoint every 5,000 steps. Consecutive checkpoints of one run differ by up to 0.035 CompBench (Section 10). The reported 118k student is the uniform average of the last five checkpoints (steps 40k, 45k, 50k, 55k and 56,974). Averaging is applied identically to every arm.

6.6 Sampling

The student is sampled with 4 Euler steps on the 4-step grid of Table 2, $w = 1$, at 512×512. Every number in this document for a distilled student comes from this setting.

7 The arms, defined once

Every trained model in this document is an instance of one objective. This section writes that objective down, names the four places where arms differ, and tabulates every arm against them, so that each results table can be read as a one-factor change.

7.1 One objective

For a caption c with candidates $j = 1, \dots, N$ (Section 6), candidate j's consistency loss is

$$\ell_c^{(j)}(\theta) = \frac{1}{K_w} \sum_{k \in \mathcal{K}} \rho\Big(\hat{x}_0\big(z^{(j)}_{k-\Delta_k}; \theta\big) - \mathrm{sg}\,\big[\tilde{x}_0^{(j),k}\big] \Big), \qquad \rho(u) = \sqrt{\|u\|_2^2 + c^2} - c, \tag{12}$$

with $z^{(j)}$ the frozen teacher's trajectory of candidate j, $\hat{x}_0$ the student's clean estimate from a noisier state, $\tilde{x}_0$ the teacher's clean estimate one step later, and Δ_k drawn afresh on every visit. Every arm minimises

$$\mathcal{L}(\theta) = \mathbb{E}_c\Big[\sum_{j=1}^{N} w_j(c)\, \ell_c^{(j)}(\theta) \Big] \; - \; \lambda\, \mathbb{E}_c\big[r_c(\theta) \big], \tag{13}$$

where the arms differ in four things and nothing else. Figure 1 shows where each enters one training update.

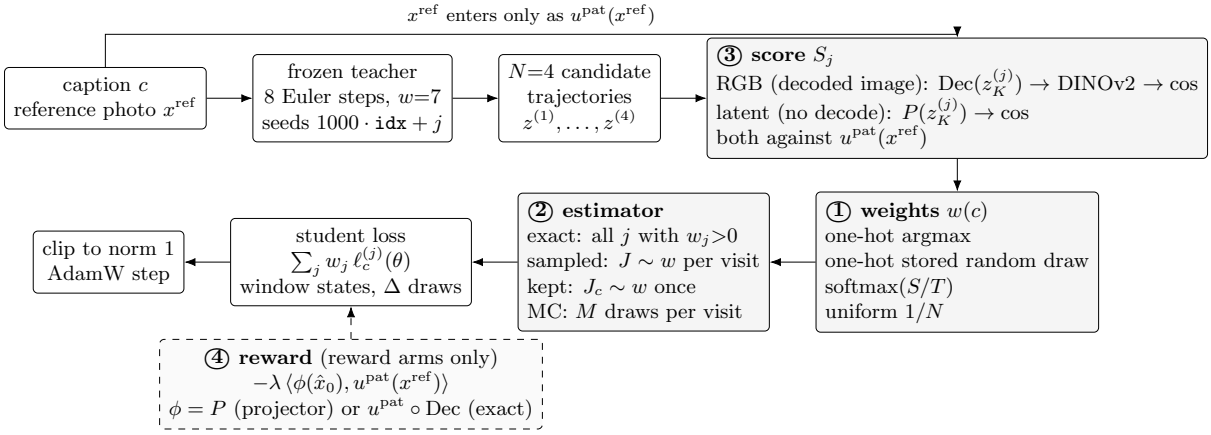

Figure 1: One training update. Every arm follows this flow; the four numbered blocks are the only places where arms differ (Table 6). The reference photograph enters only through its DINOv2 embedding, in the score and, in the reward arms, in the reward.

- **① The weights $w(c)$ over the candidates.** A function of a per-candidate score $S_j(c)$ and a temperature T:
  - *argmax*: $w_j = \mathbb{1}[j = \arg\max_l S_l]$ (the method);
  - *random*: $w_j = \mathbb{1}[j = J_c]$ with $J_c \sim \mathcal{U}\{1..N\}$ drawn once and stored in the cache (the baseline);
  - *Boltzmann*: $w_j = \pi_j = \exp(S_j/T) / \sum_l \exp(S_l/T)$, on the raw score scale;
  - *uniform*: $w_j = 1/N$ (the $T \to \infty$ limit; no selection).
- **② The estimator of the weighted sum $\sum_j w_j \ell^{(j)}$.** For one-hot weights all four coincide.

  - *exact*: every candidate with $w_j > 0$ is rolled out and its loss weighted by w_j; cost proportional to the number of candidates;
  - *sampled*: one index $J \sim w$ is drawn on each visit of the caption and $\ell^{(J)}$ is used; single-candidate cost, the same expected gradient, different optimizer updates (Section 11.4);
  - *kept*: $J_c \sim w$ is drawn once per caption and reused on every visit (the persistence control);
  - *same-caption Monte Carlo*: M independent draws per visit, losses weighted by their counts over M (implemented, not run).

- ③ **The score S_j.** Both use the reference photograph only through its DINOv2 patch-mean embedding $u^{\text{pat}}(x^{\text{ref}})$:
  - *RGB*, the decoded-image scorer of Section 5, named for reading the decoded RGB image: $S_j = \langle u^{\text{pat}}(\text{Dec}(z_K^{(j)})),\, u^{\text{pat}}(x^{\text{ref}})\rangle$, the candidate decoded and embedded with DINOv2;
  - *latent* (Section 11.5): $\hat{S}_j = \langle P(z_K^{(j)}),\, u^{\text{pat}}(x^{\text{ref}})\rangle$, the candidate's terminal latent mapped into the same space by the projector P, no decode;
  - VQAScore, DINOv2 CLS, CLIP-I, ImageReward and PickScore (Table 4) are further choices of S, used with argmax only.

- ④ **The reward $r_c(\theta)$** (Sections 11.6 and 11.7; $\lambda = 0$ everywhere else):
  - $r_c(\theta) = \frac{1}{R}\sum_{s\in\mathcal{R}} \langle \phi(\hat{x}_0(z_s^{(J)};\theta)),\, u^{\text{pat}}(x^{\text{ref}})\rangle$ on the student's own clean estimates from the $R = 2$ least-noisy supervised inputs, with the gradient flowing through the frozen map ϕ into the student; ϕ is what distinguishes the reward arms;
  - *projector, frozen*: $\phi = P$, the offline projector, throughout;
  - *projector, refreshed*: $\phi = P$, and every 100 updates P takes four steps toward the RGB scorer on decoded recent predictions, with replay of teacher candidates (Figure 2);
  - *exact*: $\phi = u^{\text{pat}} \circ \text{Dec}$, the RGB scorer itself applied to the decoded prediction inside the loop, with gradients through the VAE decoder and DINOv2 (both frozen); no projector. This is the reference the projector arms approximate (Section 11.7).

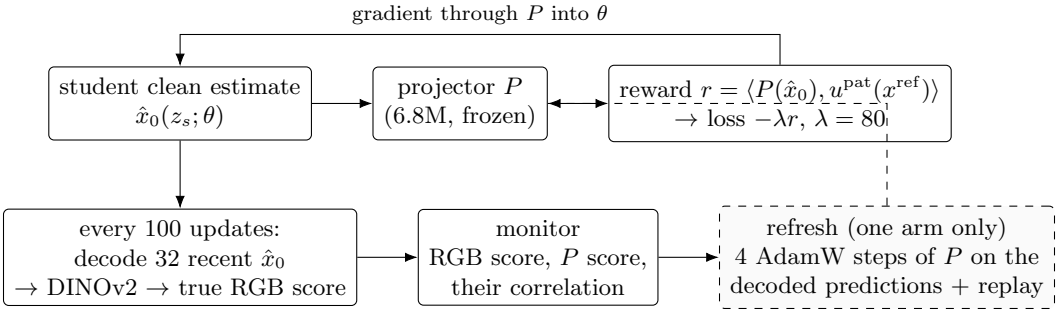

Figure 2: The reward arms of Section 11.6. Solid: the reward path used on every update. Below: the hacking monitor that decodes recent predictions and scores them with the RGB scorer, and, in the refreshed arm, the projector update it feeds.

Two further terms appear only in the ablations and are written where they are used: the real-photograph regression (Section 12) adds a pseudo-Huber term between $\hat{x}_0$ and the VAE latent of x^{ref}; the evolving-teacher variant replaces the frozen teacher in $z^{(j)}$ and $\tilde{x}_0$ by an EMA copy of the student.

7.2 Pseudocode

Algorithms 1 to 6 give every variant in executable form. Algorithm 2 is the trainer; the arms differ only in which branch of Algorithms 3, 4 and 6 it takes, and in which score column the cache carries.

Algorithm 1 Candidate cache and offline scoring (once per training pool)

Require: captions c with index `idx` and reference photograph x^{ref}; frozen teacher v_T; VAE decoder Dec; DINOv2 embedder u^{pat}; optional projector P

$u_{\text{ref}} \leftarrow u^{\text{pat}}(x^{\text{ref}})$ ▷ photo squashed to 512^2, DINOv2-B patch mean, L2-normalised
for $j = 1, \dots, N$ **do**
  $\varepsilon^{(j)} \leftarrow \mathcal{N}(0, I)$ seeded by $1000 \cdot \texttt{idx} + j$
  $z_0^{(j)}, \dots, z_K^{(j)} \leftarrow \text{Euler}_K(\varepsilon^{(j)}; v_T, c, w{=}7)$ ▷ $K = 8$; states are not stored
  $S_j \leftarrow \langle u^{\text{pat}}(\text{Dec}(z_K^{(j)})), u_{\text{ref}} \rangle$ ▷ RGB score; other scorers replace this line
  $\hat{S}_j \leftarrow \langle P(z_K^{(j)}), u_{\text{ref}} \rangle$ ▷ latent score, if P is given; no decode
end for
$J^{\text{rand}} \leftarrow \mathcal{U}\{1..N\}$ ▷ the random arm's draw, stored so every seed shares it
store (`idx`, `seed_base`$=1000 \cdot$ `idx`, $S_{1..N}$, $\hat{S}_{1..N}$, J^{rand})

Algorithm 2 One training update (all arms)

Require: cache record of caption c; student v_θ; frozen teacher v_T; window $\mathcal{K} = \{3, \dots, 7\}$; $\Delta \in \{1, 2, 3\}$; rule, estimator, score column, reward setting

$w \leftarrow \textsc{Weights}(\text{record}, \text{rule}, \text{score column}, T)$ ▷ Algorithm 3
$\mathcal{J} \leftarrow \textsc{Estimator}(w, \text{estimator}, \text{visit})$ ▷ Algorithm 4: list of (candidate, weight)
$\Delta_k \leftarrow \mathcal{U}\{1, 2, 3\}$ for $k \in \mathcal{K}$ ▷ one draw per update, shared by every candidate in $\mathcal{J}$
$g \leftarrow 0$
for $(j, \omega_j) \in \mathcal{J}$ **do**
  $z_0, \dots, z_K \leftarrow \text{Euler}_K(\varepsilon^{(j)}; v_T, c, w{=}7)$ with $\varepsilon^{(j)}$ from seed `seed_base` $+ j$ ▷ same trajectory as scored
  for $k \in \mathcal{K}$ **do** ▷ batched into one student forward
    $s \leftarrow k - \Delta_k$; $\hat{x}_0^{(k)} \leftarrow z_s - \sigma_s \, v_\theta(z_s, \sigma_s, c)$ ▷ student, one conditional pass
    $\tilde{x}_0^{(k)} \leftarrow z_k - \sigma_k \, (z_{k+1} - z_k)/(\sigma_{k+1} - \sigma_k)$ ▷ teacher target, stop-gradient
  end for
  $\ell^{(j)} \leftarrow \frac{1}{|\mathcal{K}|} \sum_k \left[\sqrt{\|\hat{x}_0^{(k)} - \tilde{x}_0^{(k)}\|^2 + c_h^2} - c_h \right]$ ▷ Equation 12
  if reward arm **then** $\ell^{(j)} \leftarrow \ell^{(j)} - \lambda \, \textsc{Reward}(\hat{x}_0^{(k)}$ for the two least-noisy $s)$ ▷ Algorithm 6
  end if
  $g \leftarrow g + \omega_j \, \nabla_\theta \ell^{(j)}$ ▷ back-propagated per candidate; peak memory of one
end for
$g \leftarrow g \cdot \min(1, \gamma / \|g\|)$; $\theta \leftarrow \text{AdamW}(\theta, g)$ ▷ $\gamma = 1$; always active
if reward arm **then** $\textsc{MonitorAndRefresh}()$ every 100 updates ▷ Algorithm 6
end if

Algorithm 3 WEIGHTS: the selection rule

Require: record with scores $S_{1..N}$ (RGB) or $\hat{S}_{1..N}$ (latent) and stored draw J^{rand}; rule; temperature T

if rule = random **then return** $w = \mathbb{1}[j = J^{\text{rand}}]$
else if rule = argmax **then return** $w = \mathbb{1}[j = \arg\max_l S_l]$
else if rule = Boltzmann **then return** $w_j = \exp\big((S_j - \max_l S_l)/T\big) \big/ \sum_l \exp\big((S_l - \max_l S_l)/T\big)$ ▷ raw scores, no z-scoring
else if rule = uniform **then return** $w_j = 1/N$
end if

Algorithm 4 ESTIMATOR: which candidates this update trains on, and with what weight

Require: weights w; estimator; selection RNG ξ (private, seeded by the run seed); caption index `idx`; map seed m

if w is one-hot **then return** $\{(\arg\max w,\, 1)\}$ ▷ all estimators coincide
else if estimator = exact **then return** $\{(j, w_j) : w_j > 0\}$ ▷ N rollouts and N student passes
else if estimator = sampled **then** $J \sim \text{Cat}(w)$ using ξ; **return** $\{(J, 1)\}$ ▷ fresh draw on every visit
else if estimator = kept **then** $J_c \sim \text{Cat}(w)$ using $\text{RNG}(m, \texttt{idx})$; **return** $\{(J_c, 1)\}$ ▷ same draw on every visit and rank
else if estimator = MC-M **then** $J_1, \ldots, J_M \overset{\text{iid}}{\sim} \text{Cat}(w)$ using ξ; **return** $\{(j,\ \#\{m : J_m = j\}/M) : \# > 0\}$
end if

Algorithm 5 Latent projector training (once, offline)

Require: 22,000 training and 2,000 validation captions, none in the pool the projector will score; teacher; Dec; u^{pat}; $\tau = 0.04$, $\lambda_{\text{rank}} = 1$

for every caption **do** ▷ data, 10 GPU shards, 7 GPU-hours
    re-roll $z_K^{(1..4)}$ from the cached seeds; $e_j \leftarrow u^{\text{pat}}(\text{Dec}(z_K^{(j)}))$; $u_{\text{ref}} \leftarrow u^{\text{pat}}(x^{\text{ref}})$; $S_j \leftarrow \langle e_j, u_{\text{ref}} \rangle$
    store ($z_K^{(j)}$ bf16, e_j, u_{ref}, $S_{1..4}$) ▷ recomputed S matches the cache to 0.002
end for
$P \leftarrow$ conv encoder $(64, 128, 256, 512, 512)$ + MLP $512{-}1024{-}768$, L2-normalised output; 6.8M parameters
for 30 epochs, batches of 32 captions **do** ▷ AdamW, one-cycle lr 3×10^{-4}, five minutes
    $\hat{e}_j \leftarrow P(z_K^{(j)})$; $\hat{S}_j \leftarrow \langle \hat{e}_j, u_{\text{ref}} \rangle$
    $\mathcal{L} \leftarrow \text{mean}_j\big[1 - \langle \hat{e}_j, e_j \rangle\big] + \lambda_{\text{rank}} \text{KL}\big(\text{softmax}(S/\tau) \,\|\, \text{softmax}(\hat{S}/\tau)\big)$
    update P; after each epoch measure top-1 agreement, Spearman, regret and VQAScore headroom on the validation captions
end for
keep the epoch with the lowest validation regret; score the target pool's cached candidates with it (Algorithm 1, line 6)

Algorithm 6 REWARD and MONITORANDREFRESH (reward arms only)

Require: score map ϕ with frozen weights, differentiable in its input: the projector P (projector arms, $\lambda = 80$) or $u^{\text{pat}} \circ \text{Dec}$, the VAE decoder followed by a differentiable resize, crop and normalisation and DINOv2 in fp32 (exact arm, $\lambda = 15.5$); u_{ref} per caption; buffer B of recent $\hat{x}_0$; RGB scorer; replay set of 2,000 teacher-candidate captions

1: **procedure** REWARD($\hat{x}_0^{(s_1)}, \hat{x}_0^{(s_2)}$) ▷ the two clean estimates from the least-noisy inputs
2: push $\hat{x}_0^{(s_1)}$ (detached) into B; **return** $\frac{1}{2}\sum_i \langle \phi(\hat{x}_0^{(s_i)}), u_{\text{ref}} \rangle$ ▷ gradient flows through ϕ into θ
3: **end procedure**
4: **procedure** MONITORANDREFRESH
5: $X \leftarrow$ the 32 most recent entries of B; $e \leftarrow u^{\text{pat}}(\text{Dec}(X))$ ▷ decode + DINOv2, forward only
6: log mean$\langle e, u_{\text{ref}} \rangle$ (true RGB score), mean$\langle P(X), u_{\text{ref}} \rangle$ (reward), and their correlation over X ▷ the hacking monitor
7: **if** refreshed arm **then**
8: **for** 4 steps **do** ▷ AdamW on P, lr 10^{-4}
9: $\mathcal{L}_P \leftarrow \text{mean}\big[1 - \langle P(X), e \rangle\big] + \mathcal{L}_{\text{replay}}$ ▷ $\mathcal{L}_{\text{replay}}$: Algorithm 5 line 8 on 8 replay captions
10: update P
11: **end for**
12: **end if**
13: **end procedure**

7.3 Every arm against the four choices

Table 6 lists every arm as its choice of weights, estimator, score and reward.

Table 6: The arms of this document. Cost is teacher rollouts and student passes per caption per update relative to the random arm (1 rollout, $K_w = 5$ student passes). Where an arm appears is given by section.

Arm (name in tables)	weights w	estimator	score S	reward	cost	where
random (fixed draw)	one-hot, stored uniform draw	–	–	–	1	everywhere (baseline)
argmax (DINOv2 patches)	one-hot on $\arg\max S$	–	RGB DINO patch	–	1	everywhere (the method)
argmax, other scorer	one-hot on $\arg\max S$	–	VQAScore, CLS, CLIP-I, ImageReward, PickScore	–	1	Sections 9, 10
Boltzmann T, exact	$\pi_j = \text{softmax}(S/T)$	exact	RGB DINO patch	–	4	Section 11
uniform, exact	$1/N$	exact	–	–	4	Section 11
Boltzmann T, sampled	π_j	sampled per visit	RGB DINO patch	–	1	Section 11
uniform, redrawn per visit	$1/N$	sampled per visit	–	–	1	Section 11
Boltzmann T, one draw kept	π_j	kept per caption	RGB DINO patch	–	1	Section 11
same-caption MC, M draws	π_j	counts/M per visit	RGB DINO patch	–	≤ 4	implemented, not run
argmax, latent scorer	one-hot on $\arg\max \hat{S}$	–	latent projector	–	1	Section 11.5
Boltzmann T exact, latent scorer	$\text{softmax}(\hat{S}/T)$	exact	latent projector	–	4	Section 11.5
argmax + projector reward, frozen	one-hot on $\arg\max S$	–	RGB DINO patch	$\lambda = 80$, $R = 2$, P frozen	1 (+P)	Section 11.6
argmax + projector reward, refreshed	one-hot on $\arg\max S$	–	RGB DINO patch	$\lambda = 80$, $R = 2$, P refreshed	1 (+P)	Section 11.6
argmax + exact DINO reward	one-hot on $\arg\max S$	–	RGB DINO patch	$\lambda = 15.5$, $R = 2$, $\phi = u^{\text{pat}} \circ \text{Dec}$	1.17 (decode in loop)	Section 11.7
118k arms; $N = 8$; batch 16	as above	–	as above	–	1	Section 10
16-step / 28-step teacher	one-hot	–	RGB DINO patch	–	1	Section 12.4
+ real-photograph term	one-hot	–	RGB DINO patch	pseudo-Huber to the photo latent	1	Section 12
evolving (EMA) teacher	one-hot on online S	–	RGB DINO patch, online	–	4 (+decode)	Section 12.6

7.4 The networks

Table 7 lists the networks involved, their sizes and roles, and which of them train.

Table 7: Every network in the pipeline, what it does, and whether it trains.

Network	Architecture and size	Role	Trained?
Teacher	SD3.5-Medium MMDiT, 2.5B, bf16	candidate rollouts ($K = 8$, $w = 7$) and targets $\tilde{x}_0$	no (frozen); EMA copy of the student in Section 12.6 only
Student	same MMDiT, initialised from the teacher, fp32 master weights	$\hat{x}_0$ at the supervised states; sampled at 4 steps, $w = 1$	yes (the only network trained in the main method)
Text encoders	CLIP-L, CLIP-G, T5-XXL	caption conditioning of both	no
VAE	SD3.5 VAE	decode candidates for scoring; decode for evaluation; in the exact-reward arm, decode $\hat{x}_0$ inside the loop with gradients passing through	no
RGB scorer	DINOv2-B, 86M; patch-mean, CLS dropped, L2-normalised	S_j (offline); the reward r_c in the exact-reward arm (gradients pass through)	no
Latent projector P	5-stage conv encoder (64, 128, 256, 512, 512; GN, SiLU), global mean pool, MLP 512–1024–768, L2-normalised; 6.8M	$\hat{S}_j$ (offline) and the reward r_c in the projector-reward arms	once, offline, on 22k held-out captions (5 min); refreshed during training in one reward arm

7.5 The projector's three settings, in the order they were tried

The latent projector P appears in three settings that build on each other; Table 8 lists them in order, with the check that sits between the second and the first.

Table 8: The three uses of the latent projector P. In every setting P is first trained once, offline, on 22,000 held-out captions (five minutes); the settings differ in what it scores and whether it moves afterwards.

	setting	what P scores, when	arms	outcome
1	**offline selector, frozen** (Section 11.5)	the cached teacher candidates $z_K^{(j)}$, once before training; the score goes into the cache and the trainer is unchanged	argmax, latent scorer; Boltzmann T=0.04 exact, latent scorer	recovers 34% of the VQAScore headroom (RGB: 44%); trained students 0.003 below their RGB counterparts, identical fidelity
–	**transfer check** (Section 11.5, R0)	the students' own 4-step samples, offline	none (diagnostic)	agreement with the RGB scorer drops (top-1 0.51 $\rightarrow$ 0.37); fine-tuning on 1,000 student latents does not restore it
2	**in-loop reward, frozen** (Section 11.6)	the student's clean estimates $\hat{x}_0$ at the two least-noisy supervised inputs, on every update; $-\lambda r$ added to the loss, $\lambda = 80$	argmax + projector reward, frozen	reward rises 0.03; true RGB score of the same predictions does not; CompBench -0.003 ± 0.005 vs argmax
3	**in-loop reward, refreshed** (Section 11.6)	as 2, and every 100 updates P takes four steps toward the RGB scorer on 32 decoded recent predictions, with replay of teacher candidates	argmax + projector reward, refreshed	reward rises 0.04; RGB score flat; agreement with the RGB scorer 0.41 vs 0.36 frozen; CompBench $+0.000 \pm 0.008$ vs argmax
4	**reference: the exact reward, no projector** (Section 11.7)	P replaced by the RGB scorer itself, decoded inside the loop with gradients through the VAE decoder and DINOv2; same predictions, same 20% gradient budget ($\lambda = 15.5$)	argmax + exact DINO reward	true RGB score of the predictions rises 0.025; CompBench $+0.011 \pm 0.006$ and GenEval2 +1.5 vs argmax, every seed; CMMD 0.80 $\rightarrow$ 0.69; held-out $\hat{x}_0$ DINO +0.008

7.6 How to read the comparisons

Figures 3 and 4 place every 3k-pool arm on one axis, Figure 5 shows each arm's seed-paired difference to argmax, Figure 6 shows what checkpoint averaging does to each arm, and Figure 7 shows where in CompBench the differences sit. Each results table in Sections 9 to 11 changes one row of Table 6 at a time; Section 7.2 gives the pseudocode, Section 11.2 the statistics used for every contrast, and Section 13 qualitative examples.

8 Evaluation

8.1 Alignment

- **T2I-CompBench.** 2,398 prompts, official scorers: BLIP-VQA for color, shape and texture; UniDet for 2D-spatial, 3D-spatial and numeracy; CLIPScore for non-spatial; the 3-in-1 score for complex. The overall score is the unweighted mean of the eight category means. One image per prompt, seed equal to the prompt index, so every arm sees identical noise.

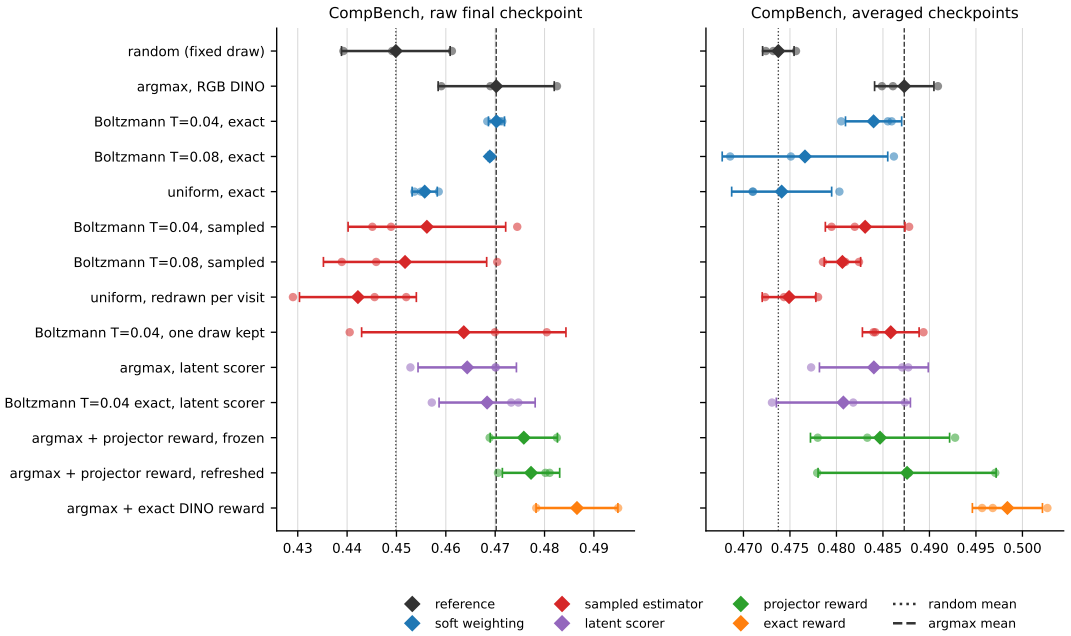

Figure 3: All arms trained on the 3k pool with the 3k configuration, three seeds each: CompBench seed values (dots), mean and standard deviation (diamond, bar), on raw final and on averaged checkpoints. Dotted and dashed lines are the random and argmax means. Colours group the arms by the choice they vary in Table 6.

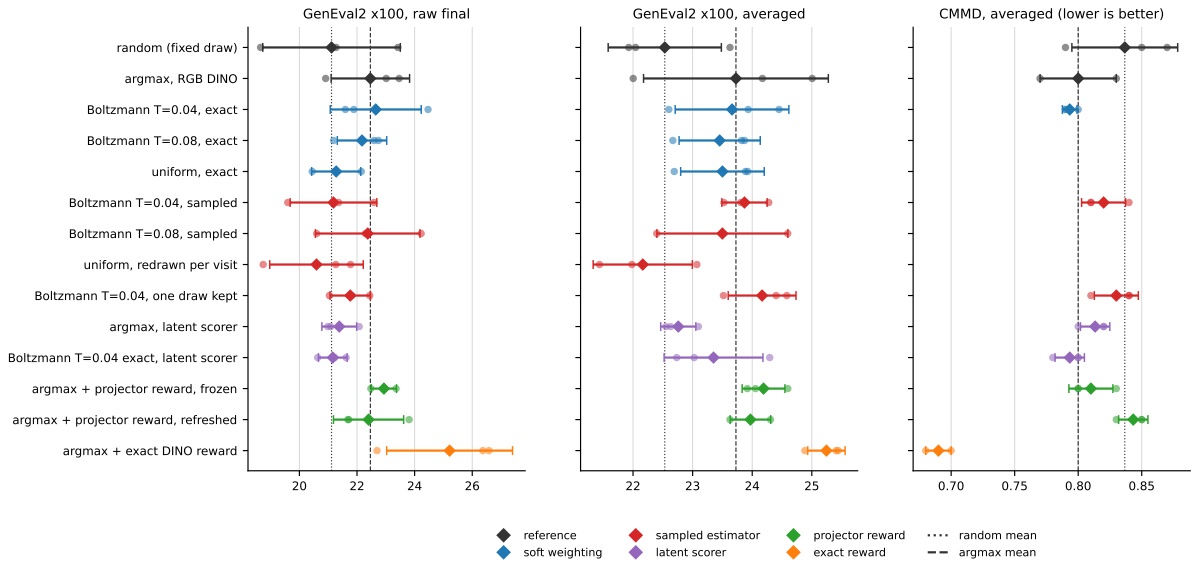

Figure 4: The same arms on GenEval2 (raw final and averaged checkpoints) and on CMMD (averaged checkpoints, lower is better). GenEval2 does not separate any two arms at three seeds; CMMD is within 0.03 for every scored arm after averaging.

- **GenEval2.** 800 prompts. Each prompt is decomposed into 3 to 10 atoms tagged object, count, attribute, position or verb. Qwen3-VL scores each atom with a probability (Soft-TIFA). The prompt score is the geometric mean of its atom scores, and the official number is the mean prompt score $\times 100$.
- **COCO VQAScore.** 5,000 held-out `val2017` captions scored by VQAScore. This is in-domain for the training captions. For the VQAScore-selected arm this metric is circular.

8.2 Fidelity

5,000 images from `val2017` captions against the 5,000 matching real photographs, centre-cropped to squares. FID uses clean-fid. CMMD uses the reference implementation: CLIP ViT-L/14 at 336 px, centre crop then bicubic resize, unit-normalised embeddings, RBF kernel with $\sigma = 10$, biased estimator, $\times 1000$. Precision and recall use k-NN manifolds. CMMD intervals are 1,000-sample bootstraps.

8.3 Statistics

Deltas are paired on identical prompts. For the three-seed experiments on the 3k pool, the interval is a hierarchical bootstrap that resamples seeds and prompts jointly (4,000 resamples). The 118k and selection-rule contrasts report per-prompt paired t-tests (Wilcoxon tests agree throughout), pooled over seeds where there are several, with the per-seed deltas alongside.

8.4 Two properties of the metrics that matter for reading the tables

GenEval2 rewards partial credit. On the geometric-mean score the 28-step teacher (17.05) scores below every 4-step student (20.7 to 23.7). On strict accuracy, the share of prompts with every atom above 0.5, the teacher leads (10.2% against 6.7 to 8.2%), and it leads on the arithmetic atom mean, on CompBench and on COCO VQAScore. The reason is that the judge gives the teacher's wrong atoms probabilities near zero, while the students' wrong atoms, mostly attribute and position atoms, get mid-range probabilities. One near-zero atom sends a geometric mean to zero, so 61% of the teacher's prompts collapse against 35 to 39% for the students, although both fail about 2.3 atoms per prompt. GenEval2 scores should therefore be read as relative numbers between students, not as evidence that a student beats the teacher.

CompBench 2D-spatial has dead prompts. The official 2D-spatial evaluator matches prompt nouns to detector labels by exact string equality. 31.8% of the official spatial prompts, and 98 of our 298, contain a noun that no label matches (for example *cow*, *sofa*, *wallet*), and they score zero for every possible image. This scales the spatial column by a constant and does not reorder models (rank correlation 0.91 between official and repaired scoring, no significant reversal). Spatial deltas are therefore understated by about 1.5× relative to other categories.

9 Results on the 3k pool

9.1 Alignment, all scorers

Table 9 is the main result. Every arm is three seeds on the same 3k pool, the same candidate cache and the same configuration.

Three points follow from the table.

- The DINOv2 patch scorer, which reads no text, recovers 60% of VQAScore's CompBench gain and 93% of its GenEval2 gain.

Table 9: 3k pool, 6,000 updates, three seeds. Absolute scores are seed means. Deltas are against the random-selection student with 95% hierarchical bootstrap intervals. Bold intervals exclude zero.

Scorer	Needs	CompBench	CompBench Δ	GenEval2	GenEval2 Δ (×100)	COCO VQAScore
random (baseline)	–	0.4584	–	20.73	–	0.8733
DINOv2 patches	photo	0.4725	**+0.0140 [+0.0051, +0.0237]**	22.89	**+2.17 [+0.24, +4.08]**	0.8850
DINOv2 CLS	photo	0.4620	+0.0036 [−0.0062, +0.0142]	21.73	+1.00 [−0.80, +2.86]	0.8799
CLIP-I	photo	0.4595	+0.0010 [−0.0112, +0.0130]	21.23	+0.51 [−1.44, +2.49]	0.8748
VQAScore	caption	0.4817	**+0.0233 [+0.0097, +0.0364]**	23.06	**+2.34 [+0.55, +4.05]**	0.8873
ImageReward	caption	0.4806	**+0.0221 [+0.0127, +0.0322]**	22.17	+1.45 [−0.43, +3.37]	0.8854
PickScore	caption	0.4713	**+0.0128 [+0.0012, +0.0245]**	22.52	+1.80 [−0.05, +3.72]	0.8850
Teacher, 8 steps, $w = 7$ (source)		0.4544		17.58		–
Teacher, 28 steps, $w = 7$		0.5053		17.05		0.9102
Base model, 4 steps, $w = 7$		0.2557		7.32		0.4762

- Mean-pooled patches beat the CLS token. Head-to-head on identical prompts the patch arm is ahead of the CLS arm by +0.0104 CompBench ($p = 10^{-5}$), +0.0108 on UniDet, +0.0154 on BLIP-VQA and +1.17 GenEval2, positive on all three seeds for every endpoint. CLIP-I, the other reference-photo scorer, is a null.
- Every 4-step student beats the 8-step teacher it was distilled from on CompBench. The 28-step teacher remains clearly better on CompBench and COCO VQAScore. Its lower GenEval2 number is the metric property of Section 8.4.

9.2 Fidelity

Table 10 shows that scored selection does not cost fidelity. Every scored arm has lower CMMD than the random baseline, and precision and recall are equal or higher. FID differences between students are within the split-half spread of about 0.5 and should not be read.

Table 10: Fidelity on 5,000 COCO `val2017` captions, 4 steps, $w = 1$, seed means over three runs. CMMD bootstrap intervals are about ±0.01.

Arm	FID ↓	CMMD ↓	Precision ↑	Recall ↑
random (baseline)	32.64	0.963	0.440	0.041
DINOv2 patches	30.72	0.893	0.465	0.052
DINOv2 CLS	30.98	0.913	0.467	0.046
CLIP-I	32.45	0.870	0.484	0.049
VQAScore	30.97	0.883	0.461	0.052
ImageReward	30.71	0.867	0.470	0.056
PickScore	31.02	0.907	0.444	0.049
Teacher, 28 steps, $w = 7$	29.21	0.640	0.549	0.156
Base model, 4 steps, $w = 7$	94.48	2.150	0.178	0.006

9.3 Absolute scores per category

Tables 11, 12 and 13 give per-category absolute scores for the teacher, the base model and the four main arms. The largest per-skill gain of the DINOv2 patch scorer on GenEval2 is on *position* (51.4 against 45.9), consistent with mean-pooled patches carrying layout, and its advantage grows with prompt complexity.

Table 11: GenEval2 (Soft-TIFA, Qwen3-VL judge), ×100, on the sealed 800-prompt pool. *Overall* is the official score, the mean over prompts of the geometric mean of a prompt's atom scores (± standard deviation over training seeds). Skill columns are the mean score of the atoms tagged with that skill. *Atom mean* is the arithmetic mean over all atoms. *Collapsed* is the share of prompts whose geometric mean falls below 0.05. Teacher and base rows are single deterministic runs. Bold marks the best 4-step distilled arm in each column.

Model	Steps / CFG	Seeds	Overall	Object	Count	Attribute	Position	Verb	Atom mean	Collapsed (%)
Teacher, 28 steps, CFG 7	28 / 7	1	17.05	87.6	44.7	66.1	45.7	17.4	64.7	60.6
Teacher, 8 steps, CFG 7 (source)	8 / 7	1	17.58	83.7	40.6	67.5	42.5	18.3	62.1	54.8
Base, 4 steps, CFG 7	4 / 7	1	7.32	43.2	20.8	49.9	26.9	4.4	36.2	79.0
Naive CD (random pick)	4 / 1	3	20.73 ± 0.94	80.5	34.8	69.5	45.9	21.4	59.8	38.5
Scored CD, DINO CLS	4 / 1	3	21.73 ± 0.76	82.7	34.4	70.1	48.5	24.2	60.9	37.0
Scored CD, DINO patches	4 / 1	3	22.89 ± 1.01	82.8	34.6	70.5	**51.4**	26.0	61.4	35.4
Scored CD, VQAScore	4 / 1	3	**23.06 ± 0.15**	**84.3**	**35.6**	**70.7**	47.8	**26.6**	**62.1**	**34.8**

Table 12: GenEval2 by prompt complexity: mean prompt score (×100) in each atom-count bucket, 100 prompts per bucket, averaged over training seeds where applicable. Bold marks the best 4-step distilled arm in each column.

Model	3	4	5	6	7	8	9	10
Teacher, 28 steps, CFG 7	36.3	32.5	22.5	19.9	12.8	4.7	5.5	2.3
Teacher, 8 steps, CFG 7 (source)	34.2	35.6	23.2	18.1	11.2	9.0	4.9	4.2
Base, 4 steps, CFG 7	21.0	8.3	8.7	6.9	3.9	4.7	3.0	2.1
Naive CD (random pick)	38.9	34.2	22.8	20.9	15.9	13.6	11.7	7.8
Scored CD, DINO CLS	43.2	34.2	24.4	22.2	15.5	13.7	13.0	7.5
Scored CD, DINO patches	41.6	**35.6**	**25.4**	**24.7**	**17.7**	**15.3**	**14.3**	8.6
Scored CD, VQAScore	**48.4**	33.4	24.6	24.1	17.3	14.0	13.2	**9.4**

10 Results at 118k captions

10.1 Weight-averaged students, three seeds

Table 14 gives the 118k result. Every arm is three training seeds on the same cache and the same configuration, and the reported weights are the uniform average of the last five checkpoints of each run. This is the largest and best-performing setting in this work.

The per-seed DINOv2 patch gains are +0.022, +0.011 and +0.019 CompBench (standard deviation 0.006, so a single-seed contrast on this branch resolves about 0.01). The gain is present in both evaluator families of CompBench: UniDet +0.026 ($p = 5 \times 10^{-9}$) and BLIP-VQA +0.017 ($p = 10^{-5}$); the CLIPScore category is a null (+0.0002). The averaged patch student is 0.021 CompBench below the 28-step teacher and 0.030 above its 8-step source.

10.2 Official protocol: ten images per prompt

T2I-CompBench's official protocol scores ten images per prompt. Table 15 repeats the evaluation of the same seven averaged models under it (images seeded by prompt and sample index, identical across arms). The gain holds on every seed with the same category profile (Table 16, Figure 8). Its size is about three quarters of the one-image number, because ten images per prompt is a less noisy estimate for both arms, and its statistical strength is far higher.

10.3 Fidelity, three seeds

Table 17 gives fidelity per seed. The 0.05 to 0.06 CMMD advantage of the scored student holds on every seed, far outside the ±0.01 bootstrap interval, and so do precision and recall. FID is

Table 13: T2I-CompBench on the sealed 2,398-prompt pool, official scorers, one image per prompt. *Mean* is the unweighted mean of the eight categories (± standard deviation over training seeds); category columns are averaged over seeds. Bold marks the best 4-step distilled arm in each column.

Model	Mean	Color	Shape	Texture	Spatial	3D-spatial	Numeracy	Non-spatial	Complex
Teacher, 28 steps, CFG 7	0.5053	0.7977	0.5732	0.7427	0.2716	0.3662	0.5915	0.3154	0.3837
Teacher, 8 steps, CFG 7 (source)	0.4544	0.7142	0.4988	0.6750	0.2425	0.3276	0.5518	0.3093	0.3163
Base, 4 steps, CFG 7	0.2557	0.4168	0.2921	0.4053	0.0582	0.1406	0.2574	0.2707	0.2048
Naive CD (random pick)	0.4584 ± 0.0075	0.7840	0.5435	0.7081	0.1783	0.2806	0.5015	0.3080	0.3634
Scored CD, DINO CLS	0.4620 ± 0.0046	0.7904	0.5388	0.6983	0.1878	0.2830	0.5205	0.3088	0.3685
Scored CD, DINO patches	0.4725 ± 0.0014	0.8041	0.5525	**0.7172**	0.2061	0.2839	0.5339	0.3093	0.3727
Scored CD, VQAScore	**0.4817 ± 0.0106**	**0.8120**	**0.5651**	0.7157	**0.2332**	**0.3063**	**0.5366**	**0.3096**	**0.3754**

Table 14: 118k captions, 56,974 updates on 4 GPUs, three training seeds, uniform average of the last five checkpoints. Per-seed scores and seed means; deltas are per-prompt paired tests against the random-selection student, pooled over seeds. The VQAScore arm has one seed and is paired against seed 0 of the baseline.

Scorer	CompBench (s0 / s1 / s2)	mean	Δ (p)	GenEval2 (s0 / s1 / s2)	mean	Δ (p)
random (baseline)	0.4642 / 0.4709 / 0.4653	0.4668	–	20.61 / 20.52 / 19.74	20.29	–
DINOv2 patches	0.4865 / 0.4821 / 0.4843	**0.4843**	+0.0175 (10^{-14})	22.05 / 21.22 / 21.48	21.58	+1.29 (0.02)
VQAScore (seed 0)	0.4827	0.4827	+0.0185 (10^{-7})	23.66	**23.66**	+3.05 (0.001)

within its split-half spread.

10.4 Why averaging is needed

Table 18 and Figure 9 evaluate every saved checkpoint of the three seed-0 runs. Consecutive checkpoints of one run differ by up to 0.035 CompBench, which is larger than the effect. The cause is the fixed-size normalised step (Section 6) with no averaging: the weights move on a plateau and each checkpoint samples a different point of it. Paired across the eight checkpoints, the patch arm is ahead of random on CompBench by +0.014 on average (7 of 8 checkpoints, $p = 0.02$) and the VQAScore arm by +0.010 (6 of 8, $p = 0.05$). Averaging the last five checkpoints raised every arm and moved the scored arms most. Over the three seeds, the raw final checkpoints give a patch-minus-random gap of only +0.007 (+0.001, +0.003, +0.017; $p = 0.003$) against +0.0175 for the averaged weights, so averaging is not optional at this scale, and it is applied identically to every arm.

The averaging also improved fidelity for every arm at seed 0: CMMD went from 0.87 to 0.84 for random, 0.86 to 0.78 for the patch arm and 0.87 to 0.79 for the VQAScore arm.

10.5 Optimizer batch size

Every update above processes four captions, one per GPU. To test whether the oscillation or the gap depends on that, both arms were retrained at seed 0 with gradient accumulation over four captions per GPU: 16 captions per update, 14,244 updates for the same two passes over the data, checkpoints every 1,250 updates so that the last-five average spans the same data window. Nothing else changed. Table 19 gives the result. The selection gap is unchanged (+0.0221 against +0.0223 for this seed at batch 4). Batch 16 lifts both arms by the same +0.005, which is not significant at one seed. It removes the oscillation of the random arm (raw final and average within 0.001) but not of the scored arm (0.012 apart), so averaging remains necessary, and batch 4 with averaging remains the reported configuration. This row is a robustness check.

Table 15: Official ten-images-per-prompt T2I-CompBench on the 118k weight-averaged models, per training seed. Paired deltas against the random-selection student.

Scorer	seed 0	seed 1	seed 2	mean, Δ (p)
random (baseline)	0.4704	0.4723	0.4729	0.4719
DINOv2 patches	0.4878	0.4838	0.4832	0.4850, +0.0131 (2×10^{-41})
VQAScore	0.4835	–	–	+0.0131 (3×10^{-18})

Table 16: CompBench per category at 118k, three-seed means of the weight-averaged students, under the one-image and the official ten-image protocol.

	one image per prompt			ten images per prompt (official)		
Category	random	DINOv2 patches	Δ	random	DINOv2 patches	Δ
color	0.7755	0.7978	+0.0223	0.7747	0.7950	+0.0203
shape	0.5338	0.5455	+0.0118	0.5367	0.5575	+0.0207
texture	0.7051	0.7233	+0.0182	0.7166	0.7260	+0.0094
2D-spatial	0.1971	0.2279	+0.0308	0.2112	0.2284	+0.0172
3D-spatial	0.2999	0.3224	+0.0224	0.3116	0.3252	+0.0136
numeracy	0.5483	0.5726	+0.0242	0.5498	0.5619	+0.0122
non-spatial	0.3141	0.3142	+0.0002	0.3137	0.3139	+0.0002
complex	0.3606	0.3707	+0.0101	0.3607	0.3717	+0.0111
mean	0.4668	0.4843	+0.0175	0.4719	0.4850	+0.0131

10.6 Number of candidates

The cache was extended from four to eight candidates per caption with the same seed convention (candidate j from seed $1000 \cdot \texttt{idx} + j$, $j = 4, \ldots, 7$), keeping the random arm's stored draw, so the $N = 4$ arms remain the exact control. Offline, best-of-eight raises the VQAScore oracle on this pool from 0.9335 to 0.9487 (headroom over the random pick +0.094 → +0.109, 16% more), and the DINOv2-selected candidate from 0.8796 to 0.8862, the same 43% of the respective headroom. Trained with argmax over eight candidates, three seeds, the averaged students score 0.4862 / 0.4859 / 0.4838 CompBench (mean 0.4853) and 21.68 / 21.44 / 21.50 GenEval2 (21.54): +0.0010 ($p = 0.64$) and −0.04 against the $N = 4$ arm, a null. Both arms are about +0.018 over random. Four candidates saturate this scorer on this teacher; the extra offline headroom of eight does not convert into a trained gain.

10.7 The exact reward at 118k captions

The exact reward (Section 11.7) was re-tested at the full 118k scale, three seeds, otherwise identical to the argmax-only 118k configuration (Table 5), $\lambda = 15.504$. Table 20 gives the result. Over argmax alone the reward adds $+0.0087 \pm 0.0047$ CompBench (every seed positive, $p = 0.08$) and $+1.45 \pm 0.80$ GenEval2; over random selection the combined effect is $+0.0262 \pm 0.0025$ CompBench ($p = 0.003$) and +2.74 GenEval2. Fidelity moves further in the same direction selection alone already moved it: CMMD falls from 0.783 (argmax) to 0.683, precision rises from 0.518 to 0.563 and recall from 0.067 to 0.087, on every seed, with FID unchanged (30.79 against 31.09). The 3k-scale result (Section 11.7: +0.0055 CompBench over argmax, CMMD 0.79 → 0.73) holds up at 40 times the caption count and roughly 10 times the update count: the reward's benefit is not an artefact of the small pool or the short schedule.

Table 17: Fidelity at 118k, weight-averaged students, 5,000 COCO `val2017` captions, 4 steps, $w = 1$, per training seed (s0 / s1 / s2).

Arm	FID ↓	CMMD ↓	Precision ↑	Recall ↑
random (baseline)	31.08 / 30.97 / 31.44	0.84 / 0.84 / 0.83	0.485 / 0.489 / 0.503	0.059 / 0.066 / 0.062
DINOv2 patches	30.98 / 30.83 / 31.47	**0.78 / 0.79 / 0.78**	**0.522 / 0.511 / 0.522**	**0.071 / 0.068 / 0.063**
VQAScore (seed 0)	30.45	0.79	0.510	0.061
Teacher, 28 steps, $w = 7$	29.21	0.64	0.549	0.156

Table 18: CompBench of every saved checkpoint of the seed-0 118k runs. The last row is the weight average of the last five checkpoints.

Step	Caption visits	random	DINOv2 patches	VQAScore	patch Δ	VQAScore Δ
5,000	20,000	0.4570	0.4793	0.4607	+0.0223	+0.0037
10,000	40,000	0.4572	0.4539	0.4515	−0.0033	−0.0057
15,000	60,000	0.4237	0.4454	0.4463	+0.0217	+0.0226
20,000	80,000	0.4342	0.4687	0.4579	+0.0345	+0.0237
30,000	120,000	0.4600	0.4612	0.4704	+0.0013	+0.0104
40,000	160,000	0.4494	0.4706	0.4721	+0.0211	+0.0227
50,000	200,000	0.4567	0.4682	0.4684	+0.0115	+0.0117
56,974	227,896	0.4573	0.4587	0.4516	+0.0014	−0.0056
average of last 5		0.4642	0.4865	0.4827	+0.0222	+0.0185

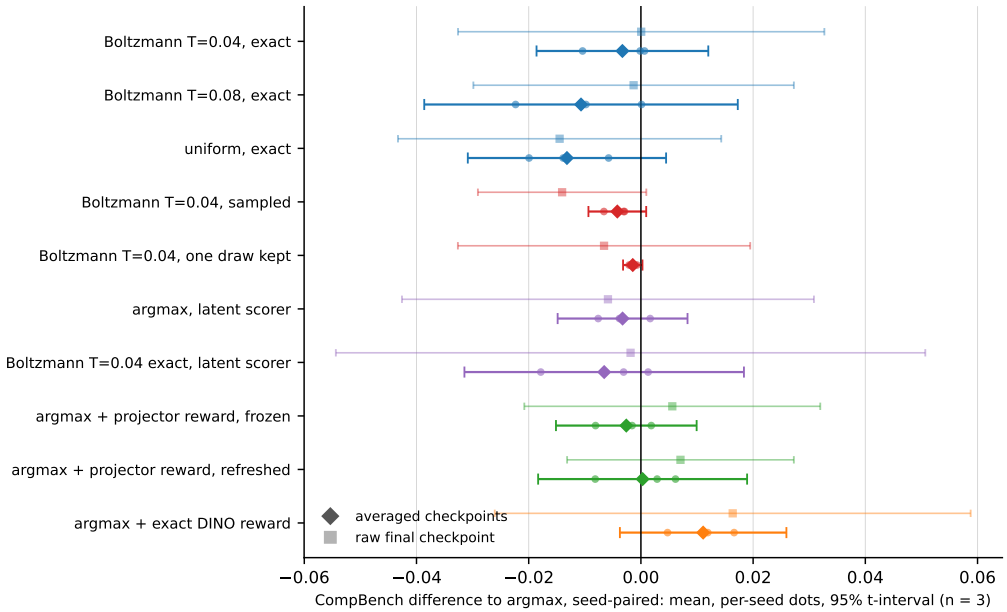

Figure 5: Seed-paired CompBench difference of every arm to argmax on averaged checkpoints (diamonds, with per-seed dots and 95% t-intervals) and on raw final checkpoints (faint squares). Intervals that cross zero are "no detected difference"; three seeds resolve about 0.01.

Table 19: Optimizer batch size at 118k, seed 0. Batch 16 uses gradient accumulation over four captions per GPU at the same data budget. CompBench / GenEval2×100.

Batch (updates)	random, raw final	random, averaged	DINOv2 patches, raw final	DINOv2 patches, averaged
4 (56,974)	0.4573 / 19.51	0.4642 / 20.61	0.4587 / 20.18	0.4865 / 22.05
16 (14,244)	0.4684 / 20.28	0.4695 / 21.22	0.4796 / 20.90	**0.4916 / 22.68**
gap at batch 16, averaged	+0.0221 CompBench ($p = 4 \times 10^{-10}$), +1.45 GenEval2 ($p = 0.10$)			

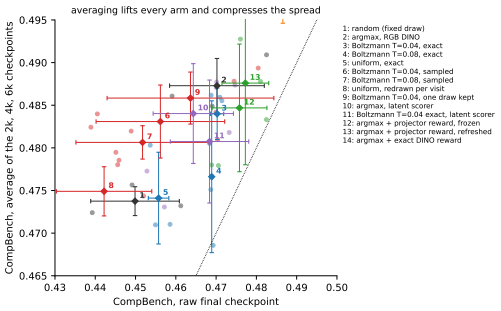

Figure 6: Raw final against averaged CompBench per arm. Averaging lifts every arm by 0.01 to 0.03 and shrinks the differences between them; the arms that gain most are those whose final iterate wanders most.

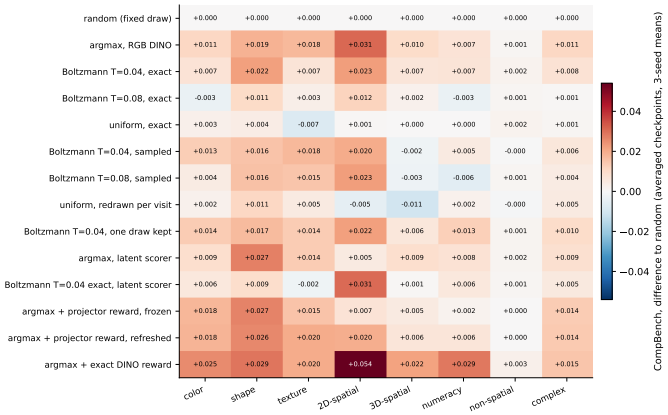

Figure 7: CompBench per category, averaged checkpoints, three-seed means, as differences to the random arm. The scored arms gain on colour, shape, spatial and numeracy and not on non-spatial, whose CLIPScore evaluator does not separate students.

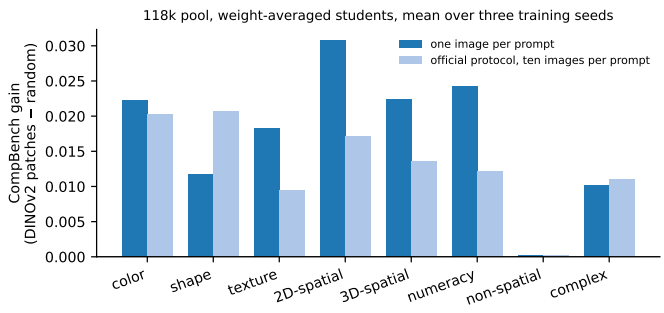

Figure 8: Per-category CompBench gain of DINOv2 patch selection over random selection at 118k (three-seed means of the averaged students) under both protocols. The gain is spread over the attribute-binding, spatial and counting categories and is zero on non-spatial, whose CLIPScore evaluator does not separate any two students in this work.

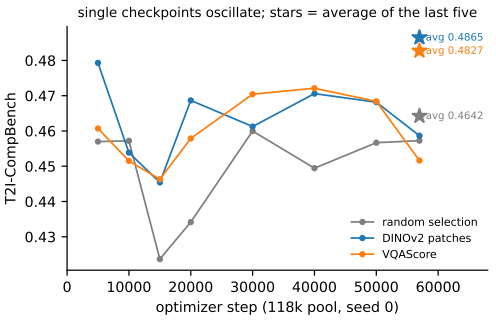

Figure 9: CompBench of every saved checkpoint of the seed-0 118k runs; the star marks the weight average of the last five checkpoints of each arm (Section 10.4).

Table 20: The exact reward at 118k captions, 56,974 updates, three training seeds, weight-averaged students. Fidelity: 5,000 COCO `val2017` captions, seed means.

Arm	CompBench	Δ vs. argmax	GenEval2	CMMD ↓	Precision	Recall
Random selection	0.4668	–	20.29	0.837	0.492	0.062
Argmax, no reward	0.4843	–	21.58	0.783	0.518	0.067
Argmax + exact reward	0.4930 ± 0.0033	+0.0087 ± 0.0047 (p=0.08)	23.03 ± 1.15	0.683	0.563	0.087

11 Selection rule: argmax, Boltzmann weighting and sampling

Argmax selection is one point on a family of rules; Table 6 defines every arm of this section against the objective of Equation 13, and Figures 3 to 7 compare them all at once. The natural alternative to argmax weights the candidates by a Boltzmann distribution over the raw scores,

$$\pi_j = \frac{\exp(S_j/T)}{\sum_{l=1}^{4} \exp(S_l/T)}, \tag{14}$$

which recovers argmax as $T \to 0$ and uniform weighting as $T \to \infty$. Two estimators of the weighted objective $\sum_j \pi_j \mathcal{L}_j$ were trained. *Exact* weighting rolls out all four candidates and back-propagates each candidate's loss with weight π_j: four teacher rollouts and four student passes per caption, 3.1 times the wall-clock of the single-candidate trainer (211 against 69 minutes per 3k run, measured on the same GPUs on one H200), at the same peak memory because candidates are back-propagated one at a time. *Sampled* weighting draws one candidate $J \sim \pi$ on every visit of the caption and trains on it alone at single-candidate cost. Every candidate is evaluated at its own trajectory, so neither estimator regresses onto an averaged target; the two have the same expected gradient, $\mathbb{E}[g_J] = \sum_j \pi_j g_j = g_F$, but not the same optimizer update (Section 11.4). Three controls complete the design: the random baseline, which draws one candidate per caption uniformly and keeps it; a *uniform-visit* arm that redraws uniformly on every visit; and a *frozen* Boltzmann arm that draws $J_c \sim \pi_c$ once per caption and keeps it, which has the sampled arm's selection distribution without per-visit resampling. A same-caption Monte-Carlo estimator (M independent draws per visit, losses weighted by their counts, one clipped step) is implemented in the release code and was not run.

11.1 Selection entropy and effective sample size

The temperature is calibrated on the scores themselves. Figure 10 and Table 21 summarise the DINOv2 patch score over the four candidates of a caption. Within a caption the scores have a standard deviation of 0.070 (range 0.18, margin between the best and the second-best candidate 0.05); between captions the mean score varies twice as much (0.14), which is why scores are only ever compared within a caption. Two effective-count summaries of the weights are used and they are not interchangeable: the entropy count $N_H = \exp H(\pi)$ and the Kish count $N_K = 1/\sum_j \pi_j^2$. At $T = 0.04$ the median N_H is 2.5 and the median N_K is 2.2, with a mean argmax weight of 0.63; at $T = 0.08$ they are 3.3 and 3.0. The Kish count is the one tied to resampling: the probability that two independent visits train on different candidates is $1 - \sum_j \pi_j^2 = 1 - 1/N_K$, 0.49 at $T = 0.04$ and 0.63 at $T = 0.08$, and in the two visits per caption of the 3k configuration the sampled arm sees on average 1.48 (1.63) distinct candidates of the four that the exact arm sees on every visit. The 3k and 118k pools have the same statistics, so a temperature calibrated on one transfers to the other. Stronger teachers compress the candidates: on the 16-step $w = 4.5$ and 28-step caches the within-caption spread almost halves (0.041 and 0.038) and the entropy at a fixed temperature rises (0.75 and 0.76 at $T = 0.04$). This is the offline face of the vanishing selection gain of Section 12.4.

11.2 Protocol and statistics

All arms were trained on the 3k pool with the 3k configuration, three seeds each, from one frozen copy of the trainer; the random and argmax arms were retrained alongside the new ones so that every contrast is a one-factor change. Their absolute level is 0.008 below the earlier runs of Table 9 (a different launch, within the seed spread), so contrasts are read within this section. Every arm is evaluated under two protocols: the raw final checkpoint, and the uniform average of the checkpoints saved at 2,000, 4,000 and 6,000 updates, applied with the same rule to every arm. The averaging is the 3k analogue of the last-five average that the 118k results

Table 21: DINOv2 patch score statistics per candidate cache ($N = 4$): within-caption standard deviation, mean range, mean margin between the two best candidates, and the mean normalised entropy and median entropy count N_H of the Boltzmann weights at the two temperatures used in training. On the 3k cache the median Kish count N_K is 2.2 at $T = 0.04$ and 3.0 at $T = 0.08$.

					$T = 0.04$		$T = 0.08$	
Cache (teacher)	captions	within std	range	top-2 margin	entropy	N_H	entropy	N_H
3k, 8 steps, $w = 7$	3,000	0.069	0.175	0.049	0.63	2.54	0.82	3.33
118k, 8 steps, $w = 7$	113,948	0.070	0.179	0.051	0.62	2.52	0.81	3.32
3k, 16 steps, $w = 4.5$	3,000	0.041	0.105	0.034	0.75	3.01	0.91	3.65
3k, 28 steps, $w = 7$	3,000	0.038	0.099	0.033	0.76	3.09	0.91	3.68

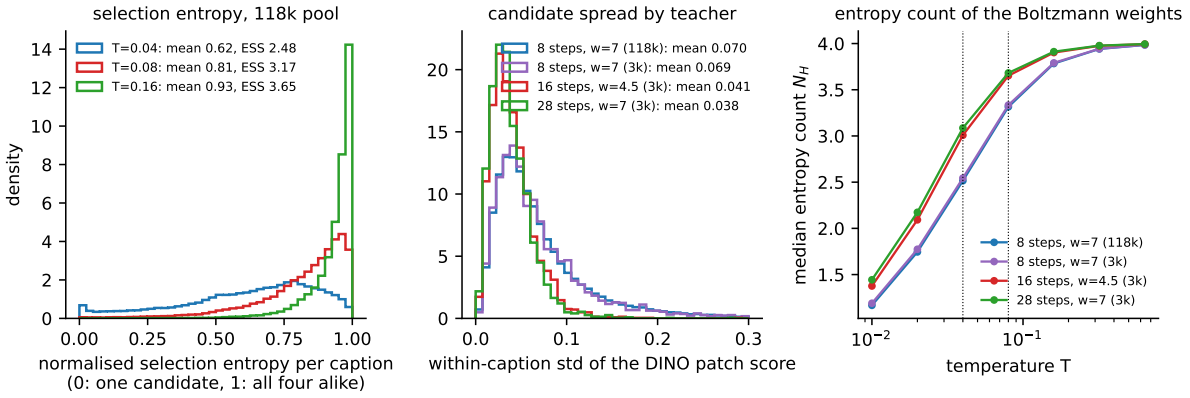

Figure 10: Left: distribution over captions of the normalised selection entropy at three temperatures (118k pool). Middle: within-caption spread of the DINOv2 patch score for the four caches; stronger teachers leave less to select on. Right: median entropy count N_H of the Boltzmann weights as a function of temperature; the two temperatures used in training are marked.

require (Section 10.4); it raises every arm by 0.014 to 0.027 CompBench, and it matters for the comparison below because arms differ in how much their final iterate wanders.

Uncertainty is reported at the level of training runs. For a contrast between arms A and B the three seed-paired differences (seed s of A minus seed s of B; at equal seed the arms share the caption order and the Δ draws) give a mean, a standard deviation, a 95% t-interval and a one-sample t-test with two degrees of freedom. Three seeds resolve a difference of about 0.01 CompBench and nothing on GenEval2, whose seed spread is 1 to 2 points. The prompt-level p-values of the paired tests used elsewhere in this document describe uncertainty over prompts for the seed-mean model, not over repeated training, and are not used to compare selection rules. A non-significant contrast is reported as "no detected improvement"; none of the tests here is an equivalence test.

11.3 Result

Table 22 (per-arm scores), Tables 23 and 24 (seed-paired contrasts on raw and averaged checkpoints) and Figure 11 give the result.

- **Argmax over random is the one contrast established at the level of training runs:** $+0.0203 \pm 0.0008$ on raw checkpoints ($p = 0.001$) and $+0.0135 \pm 0.0042$ after averaging ($p = 0.03$).
- **Exact Boltzmann weighting shows no detected improvement over argmax** at either temperature: $T = 0.04$ is $+0.0001 \pm 0.013$ raw and -0.003 ± 0.006 averaged; $T = 0.08$

Table 22: Selection rule, 3k pool, three seeds. CompBench per seed (s0 / s1 / s2) and mean ± standard deviation over seeds, for the raw final checkpoint and for the uniform average of the checkpoints at 2k, 4k and 6k updates; GenEval2×100 seed means under both protocols; GPU-hours per training run on one H200. *exact*: every candidate's loss weighted by π_j; *sampled*: one draw from π per visit; *kept*: one draw from π per caption, fixed for the run.

	CompBench, raw final		CompBench, averaged		GenEval2		
Rule	s0 / s1 / s2	mean ± sd	s0 / s1 / s2	mean ± sd	raw	avg.	GPU-h
random (fixed draw)	0.4491 / 0.4393 / 0.4612	0.4499 ± 0.0110	0.4756 / 0.4724 / 0.4732	0.4738 ± 0.0017	21.12	22.53	1.2
argmax	0.4690 / 0.4591 / 0.4825	**0.4702 ± 0.0118**	0.4849 / 0.4861 / 0.4909	**0.4873 ± 0.0032**	22.46	23.73	1.2
Boltzmann T=0.04, exact	0.4714 / 0.4709 / 0.4684	0.4703 ± 0.0016	0.4855 / 0.4859 / 0.4805	0.4840 ± 0.0030	22.65	23.66	3.5
Boltzmann T=0.08, exact	0.4687 / 0.4688 / 0.4693	0.4689 ± 0.0003	0.4751 / 0.4862 / 0.4686	0.4766 ± 0.0089	22.17	23.45	3.6
uniform ($T \to \infty$), exact	0.4586 / 0.4536 / 0.4550	0.4557 ± 0.0025	0.4710 / 0.4803 / 0.4710	0.4741 ± 0.0054	21.28	23.50	3.6
Boltzmann T=0.04, sampled	0.4489 / 0.4451 / 0.4745	0.4562 ± 0.0160	0.4820 / 0.4795 / 0.4878	0.4831 ± 0.0043	21.18	23.87	1.2
Boltzmann T=0.08, sampled	0.4459 / 0.4389 / 0.4704	0.4517 ± 0.0165	0.4785 / 0.4824 / 0.4810	0.4806 ± 0.0020	22.37	23.50	1.2
uniform-visit (redraw per visit)	0.4520 / 0.4290 / 0.4456	0.4422 ± 0.0118	0.4744 / 0.4723 / 0.4780	0.4749 ± 0.0029	20.59	22.16	1.2
Boltzmann T=0.04, one draw per caption, kept	0.4699 / 0.4405 / 0.4804	0.4636 ± 0.0207	0.4842 / 0.4840 / 0.4894	0.4858 ± 0.0030	21.77	24.17	1.2

Table 23: Seed-paired CompBench contrasts between selection rules, raw final checkpoints (seed s of the first arm minus seed s of the second; the arms share caption order and Δ draws at equal seed): per-seed differences, mean ± standard deviation over seeds, 95% t-interval and the seed-level t-test with $n = 3$.

Contrast	per seed	mean ± sd [95% CI] (p)
against the random baseline		
argmax vs random (fixed draw)	+0.0199 / +0.0198 / +0.0213	$+0.0203 \pm 0.0008$ [+0.018, +0.022] ($p = 0.001$)
Boltzmann T=0.04, exact vs random (fixed draw)	+0.0223 / +0.0317 / +0.0072	$+0.0204 \pm 0.0124$ [−0.010, +0.051] ($p = 0.10$)
Boltzmann T=0.08, exact vs random (fixed draw)	+0.0195 / +0.0295 / +0.0080	$+0.0190 \pm 0.0107$ [−0.008, +0.046] ($p = 0.09$)
uniform ($T \to \infty$), exact vs random (fixed draw)	+0.0094 / +0.0144 / -0.0063	$+0.0058 \pm 0.0108$ [−0.021, +0.033] ($p = 0.45$)
Boltzmann T=0.04, sampled vs random (fixed draw)	-0.0002 / +0.0059 / +0.0132	$+0.0063 \pm 0.0067$ [−0.010, +0.023] ($p = 0.25$)
Boltzmann T=0.08, sampled vs random (fixed draw)	-0.0032 / -0.0003 / +0.0092	$+0.0019 \pm 0.0065$ [−0.014, +0.018] ($p = 0.67$)
uniform-visit (redraw per visit) vs random (fixed draw)	+0.0028 / -0.0102 / -0.0157	-0.0077 ± 0.0095 [−0.031, +0.016] ($p = 0.30$)
Boltzmann T=0.04, one draw per caption, kept vs random (fixed draw)	+0.0208 / +0.0013 / +0.0192	$+0.0138 \pm 0.0109$ [−0.013, +0.041] ($p = 0.16$)
against argmax		
Boltzmann T=0.04, exact vs argmax	+0.0024 / +0.0119 / -0.0141	$+0.0001 \pm 0.0131$ [−0.033, +0.033] ($p = 1.00$)
Boltzmann T=0.08, exact vs argmax	-0.0004 / +0.0097 / -0.0132	-0.0013 ± 0.0115 [−0.030, +0.027] ($p = 0.86$)
uniform ($T \to \infty$), exact vs argmax	-0.0105 / -0.0054 / -0.0276	-0.0145 ± 0.0116 [−0.043, +0.014] ($p = 0.16$)
Boltzmann T=0.04, sampled vs argmax	-0.0201 / -0.0140 / -0.0080	-0.0140 ± 0.0060 [−0.029, +0.001] ($p = 0.06$)
Boltzmann T=0.04, one draw per caption, kept vs argmax	+0.0009 / -0.0186 / -0.0021	-0.0066 ± 0.0105 [−0.033, +0.019] ($p = 0.39$)
estimator and persistence		
Boltzmann T=0.04, exact vs Boltzmann T=0.04, sampled	+0.0225 / +0.0258 / -0.0061	$+0.0141 \pm 0.0175$ [−0.029, +0.058] ($p = 0.30$)
Boltzmann T=0.04, one draw per caption, kept vs Boltzmann T=0.04, sampled	+0.0210 / -0.0046 / +0.0060	$+0.0075 \pm 0.0129$ [−0.025, +0.039] ($p = 0.42$)
uniform ($T \to \infty$), exact vs uniform-visit (redraw per visit)	+0.0066 / +0.0246 / +0.0094	$+0.0135 \pm 0.0097$ [−0.011, +0.038] ($p = 0.14$)

is -0.001 ± 0.012 raw and -0.011 ± 0.011 averaged. The averaged $T = 0.04$ interval is ± 0.015, so a soft rule within 0.01 of argmax is not excluded; one that is better is not supported, and it costs 2.3 times the compute.

- **Score-informed weighting is what matters.** At the same exact compute, $T = 0.04$ is ahead of uniform weighting by 0.0145 on raw checkpoints (positive on every seed) and by +0.010 (per seed +0.015, +0.006, +0.010) after averaging, and uniform weighting of all four candidates is within noise of the random baseline under both protocols. Exposure to more trajectories per caption is worth little on its own.

- **The sampled estimator's deficit is mostly a property of its final iterate.** On raw checkpoints the sampled $T = 0.04$ arm is 0.014 ± 0.006 below argmax and 0.014 ± 0.018 below exact weighting; after averaging the gaps are 0.004 ± 0.002 and -0.001 ± 0.007. The sampled arm's raw checkpoints vary four times more across seeds than its averaged weights (0.016 against 0.004). What remains after averaging is of the size the optimizer-level differences of Section 11.4 can produce and is not resolved at three seeds.

- **Persistence of the selected candidate is not established as a separate effect.** Redrawing uniformly on every visit is 0.008 ± 0.010 below the fixed random draw on raw checkpoints but $+0.001 \pm 0.003$ after averaging. The frozen Boltzmann arm, which

Table 24: Seed-paired CompBench contrasts between selection rules, averaged checkpoints (2k, 4k and 6k updates) (seed s of the first arm minus seed s of the second; the arms share caption order and Δ draws at equal seed): per-seed differences, mean $\pm$ standard deviation over seeds, 95% t-interval and the seed-level t-test with $n = 3$.

Contrast	per seed	mean $\pm$ sd [95% CI] (p)
against the random baseline		
argmax vs random (fixed draw)	+0.0092 / +0.0137 / +0.0177	$+0.0135 \pm 0.0042$ [+0.003, +0.024] ($p = 0.03$)
Boltzmann T=0.04, exact vs random (fixed draw)	+0.0099 / +0.0135 / +0.0073	$+0.0102 \pm 0.0031$ [+0.002, +0.018] ($p = 0.03$)
Boltzmann T=0.08, exact vs random (fixed draw)	-0.0006 / +0.0138 / -0.0046	$+0.0029 \pm 0.0097$ [−0.021, +0.027] ($p = 0.66$)
uniform ($T \to \infty$), exact vs random (fixed draw)	-0.0046 / +0.0079 / -0.0022	$+0.0004 \pm 0.0067$ [−0.016, +0.017] ($p = 0.93$)
Boltzmann T=0.04, sampled vs random (fixed draw)	+0.0063 / +0.0071 / +0.0146	$+0.0093 \pm 0.0046$ [−0.002, +0.021] ($p = 0.07$)
Boltzmann T=0.08, sampled vs random (fixed draw)	+0.0029 / +0.0100 / +0.0077	$+0.0069 \pm 0.0036$ [−0.002, +0.016] ($p = 0.08$)
uniform-visit (redraw per visit) vs random (fixed draw)	-0.0013 / -0.0001 / +0.0048	$+0.0012 \pm 0.0032$ [−0.007, +0.009] ($p = 0.60$)
Boltzmann T=0.04, one draw per caption, kept vs random (fixed draw)	+0.0085 / +0.0116 / +0.0162	$+0.0121 \pm 0.0038$ [+0.003, +0.022] ($p = 0.03$)
against argmax		
Boltzmann T=0.04, exact vs argmax	+0.0006 / -0.0001 / -0.0104	-0.0033 ± 0.0062 [−0.019, +0.012] ($p = 0.45$)
Boltzmann T=0.08, exact vs argmax	-0.0098 / +0.0001 / -0.0223	-0.0107 ± 0.0112 [−0.039, +0.017] ($p = 0.24$)
uniform ($T \to \infty$), exact vs argmax	-0.0139 / -0.0058 / -0.0199	-0.0132 ± 0.0071 [−0.031, +0.004] ($p = 0.08$)
Boltzmann T=0.04, sampled vs argmax	-0.0029 / -0.0066 / -0.0031	-0.0042 ± 0.0021 [−0.009, +0.001] ($p = 0.07$)
Boltzmann T=0.04, one draw per caption, kept vs argmax	-0.0007 / -0.0021 / -0.0016	-0.0015 ± 0.0007 [−0.003, +0.000] ($p = 0.07$)
estimator and persistence		
Boltzmann T=0.04, exact vs Boltzmann T=0.04, sampled	+0.0036 / +0.0065 / -0.0073	$+0.0009 \pm 0.0073$ [−0.017, +0.019] ($p = 0.85$)
Boltzmann T=0.04, one draw per caption, kept vs Boltzmann T=0.04, sampled	+0.0022 / +0.0045 / +0.0015	$+0.0028 \pm 0.0016$ [−0.001, +0.007] ($p = 0.09$)
uniform ($T \to \infty$), exact vs uniform-visit (redraw per visit)	-0.0033 / +0.0080 / -0.0070	-0.0008 ± 0.0078 [−0.020, +0.019] ($p = 0.88$)

Table 25: Fidelity of the T=0.04 selection-rule arms, the latent-scorer arms and the reward arms (Sections 11.6 and 11.7) on 5,000 COCO `val2017` captions, 4 steps, $w = 1$: per seed (s0 / s1 / s2) and seed mean in brackets, for the raw final checkpoints and for the averaged checkpoints. CMMD bootstrap intervals are ± 0.01 per model; FID differences below its split-half spread of about 0.5 should not be read.

Rule	checkpoint	FID ↓	CMMD ↓	Precision ↑	Recall ↑
random (fixed draw)	raw final	32.91 / 33.16 / 31.17 (32.41)	0.91 / 0.96 / 0.97 (0.95)	0.456 / 0.436 / 0.414 (0.435)	0.045 / 0.046 / 0.037 (0.043)
argmax	raw final	31.90 / 30.57 / 29.62 (30.70)	0.89 / 0.94 / 0.83 (0.89)	0.455 / 0.450 / 0.485 (0.463)	0.052 / 0.047 / 0.060 (0.053)
Boltzmann T=0.04, exact	raw final	30.97 / 29.43 / 31.22 (30.54)	0.85 / 0.87 / 0.85 (0.86)	0.488 / 0.461 / 0.504 (0.484)	0.058 / 0.051 / 0.050 (0.053)
Boltzmann T=0.04, sampled	raw final	32.90 / 32.01 / 31.40 (32.10)	0.90 / 1.02 / 0.93 (0.95)	0.464 / 0.434 / 0.449 (0.449)	0.040 / 0.040 / 0.043 (0.041)
Boltzmann T=0.04, one draw per caption, kept	raw final	32.64 / 33.32 / 32.37 (32.78)	0.89 / 0.98 / 0.90 (0.92)	0.445 / 0.444 / 0.443 (0.444)	0.052 / 0.039 / 0.047 (0.046)
latent scorer, argmax	raw final	33.09 / 31.08 / 31.66 (31.94)	0.85 / 0.90 / 0.91 (0.89)	0.507 / 0.429 / 0.449 (0.462)	0.053 / 0.048 / 0.042 (0.048)
latent scorer, Boltzmann T=0.04, exact	raw final	31.79 / 31.60 / 31.23 (31.54)	0.81 / 0.83 / 0.85 (0.83)	0.512 / 0.492 / 0.497 (0.500)	0.056 / 0.057 / 0.053 (0.055)
argmax + projector reward, frozen	raw final	30.90 / 30.59 / 29.16 (30.22)	0.83 / 0.87 / 0.84 (0.85)	0.502 / 0.455 / 0.477 (0.478)	0.054 / 0.060 / 0.058 (0.057)
argmax + projector reward, refreshed	raw final	31.19 / 31.51 / 30.90 (31.20)	0.85 / 0.95 / 0.90 (0.90)	0.524 / 0.426 / 0.466 (0.472)	0.048 / 0.050 / 0.054 (0.051)
argmax + exact DINO reward	raw final	30.43 / 29.70 / 30.66 (30.26)	0.73 / 0.74 / 0.84 (0.77)	0.538 / 0.531 / 0.477 (0.515)	0.074 / 0.075 / 0.058 (0.069)
random (fixed draw)	averaged	30.58 / 30.45 / 30.24 (30.42)	0.79 / 0.85 / 0.87 (0.84)	0.507 / 0.474 / 0.457 (0.479)	0.056 / 0.055 / 0.051 (0.054)
argmax	averaged	30.53 / 29.85 / 29.99 (30.12)	0.83 / 0.80 / 0.77 (0.80)	0.489 / 0.495 / 0.484 (0.489)	0.062 / 0.067 / 0.070 (0.066)
Boltzmann T=0.04, exact	averaged	30.37 / 29.81 / 30.75 (30.31)	0.79 / 0.80 / 0.79 (0.79)	0.508 / 0.489 / 0.507 (0.501)	0.066 / 0.069 / 0.059 (0.065)
Boltzmann T=0.04, sampled	averaged	29.83 / 30.21 / 30.86 (30.30)	0.81 / 0.84 / 0.81 (0.82)	0.484 / 0.457 / 0.486 (0.476)	0.062 / 0.059 / 0.062 (0.061)
Boltzmann T=0.04, one draw per caption, kept	averaged	30.98 / 31.11 / 31.05 (31.05)	0.84 / 0.81 / 0.84 (0.83)	0.478 / 0.484 / 0.474 (0.479)	0.062 / 0.066 / 0.061 (0.063)
latent scorer, argmax	averaged	31.01 / 29.84 / 30.71 (30.52)	0.82 / 0.80 / 0.82 (0.81)	0.495 / 0.475 / 0.485 (0.485)	0.061 / 0.062 / 0.057 (0.060)
latent scorer, Boltzmann T=0.04, exact	averaged	30.23 / 30.68 / 30.47 (30.46)	0.78 / 0.80 / 0.80 (0.79)	0.512 / 0.493 / 0.509 (0.505)	0.067 / 0.066 / 0.067 (0.067)
argmax + projector reward, frozen	averaged	30.10 / 29.82 / 29.06 (29.66)	0.83 / 0.80 / 0.80 (0.81)	0.492 / 0.469 / 0.501 (0.487)	0.061 / 0.062 / 0.063 (0.062)
argmax + projector reward, refreshed	averaged	30.04 / 30.05 / 29.39 (29.83)	0.85 / 0.85 / 0.83 (0.84)	0.496 / 0.460 / 0.479 (0.478)	0.055 / 0.061 / 0.064 (0.060)
argmax + exact DINO reward	averaged	29.96 / 30.67 / 30.57 (30.40)	0.68 / 0.69 / 0.70 (0.69)	0.543 / 0.544 / 0.535 (0.541)	0.089 / 0.088 / 0.086 (0.088)

keeps one draw from π per caption, is $+0.008 \pm 0.013$ above the resampled arm on raw checkpoints and $+0.003 \pm 0.002$ after averaging (positive on all three seeds, $p = 0.09$), and 0.0015 ± 0.0007 below argmax after averaging. Its raw checkpoints vary the most of all arms across seeds (0.021), since each seed also carries its own selection map; averaging removes that as well. Switching candidates changes the input trajectory and its target together, and in the two visits per caption of this configuration the sampled arm sees 1.5 distinct candidates on average, so target churn, candidate coverage and optimization variance are not separated by these controls.

- **Stability.** On raw checkpoints the exact arms vary far less across seeds than the single-candidate arms (standard deviation 0.0016 and 0.0003 against 0.011 to 0.017); after averaging the difference is gone (0.003 and 0.009 against 0.002 to 0.004). Averaging four trajectories per update stabilises the iterate, not the solution.

- **Fidelity.** Table 25 gives FID, CMMD, precision and recall for the T=0.04 arms. On raw final checkpoints the exact arm has the best fidelity (CMMD 0.86 against 0.89 for argmax

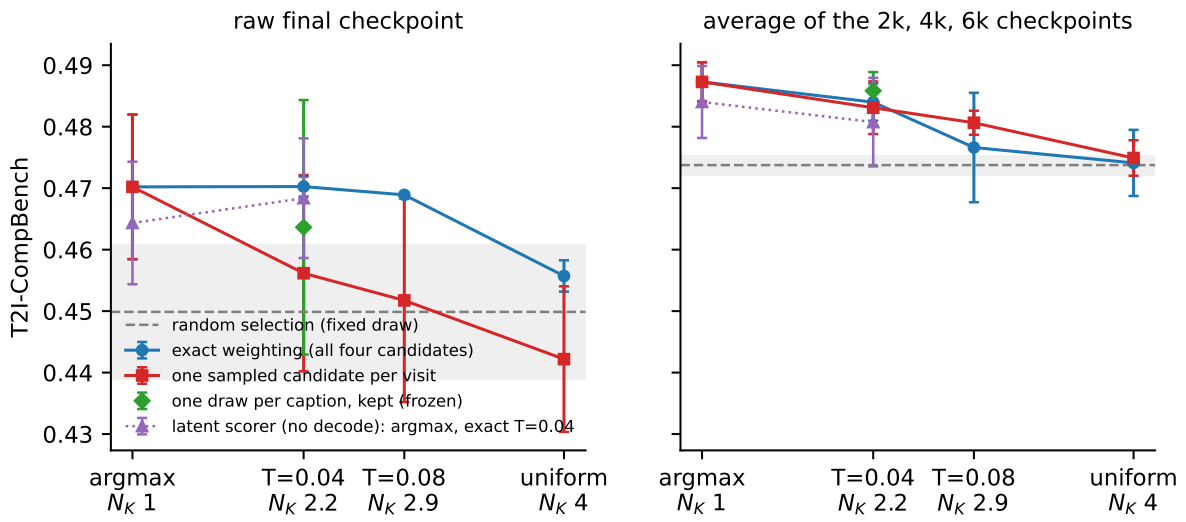

Figure 11: CompBench of the selection rules as a function of the Kish count of their weights, raw final checkpoints (left) and the average of the 2k, 4k and 6k checkpoints (right). Points are seed means, bars are seed standard deviations, the grey band is the random baseline. On raw checkpoints the sampled arms fall away from argmax while the exact arms stay flat; after identical averaging the two estimators coincide and the ordering follows the weight on the best candidate.

and 0.95 for both random and the resampled arm) and the resampled arm sits at the random level; after averaging every arm improves by about 0.1 CMMD and the differences shrink to within the seed spread (exact minus argmax -0.007 ± 0.031, resampled minus argmax $+0.020 \pm 0.035$, frozen minus argmax $+0.030 \pm 0.035$). The one fidelity contrast that keeps its sign on every seed after averaging is exact against resampled, -0.027 ± 0.012 CMMD and $+0.026$ precision ($p = 0.06$): weighting all four candidates gives a slightly better distribution match than sampling one, with no alignment difference.

- **GenEval2** does not separate any two selection rules at three seeds.

11.4 Gradient diagnostic

Equal expected gradients do not give equal optimizer updates. The trainer clips the global gradient norm to $\gamma = 1$ and then applies AdamW, and clipping does not commute with expectation: $\mathbb{E}[C(g_J)] \neq C(\mathbb{E}[g_J])$ for $C(g) = g \min(1, \gamma/\|g\|)$, while Adam's second moment sees $\mathbb{E}[g_J \odot g_J] = g_F \odot g_F + \mathrm{Var}(g_J)$. To measure how large these effects are for this loss, all four candidate gradients of a caption were computed at two frozen model states, the pretrained initialisation and the argmax student of seed 0, on 48 held-out captions of the 118k pool (none in the 3k pool), with one Δ draw shared by the four candidates. Everything in Table 26 is a function of the 4×4 Gram matrix of the candidate gradients and π at $T = 0.04$:

$$V_c = \sum_j \pi_j \|g_j - g_F\|^2, \qquad B_c = \Big\| \sum_j \pi_j C(g_j) - C(g_F) \Big\|. \tag{15}$$

Two references are computed per caption: the argmax candidate's gradient under a second Δ draw on the same trajectory, which is the gradient noise every arm carries, and the argmax candidate rolled out alone instead of in the four-candidate batch, which checks that batch shape is not a confound.

Three things follow. The candidate gradients of one caption are only weakly aligned (mean pairwise cosine 0.17 at the trained state), so the one-sample estimator carries a relative variance

Table 26: Gradient diagnostic at $T = 0.04$, 48 held-out captions, means over captions (medians in brackets). Every gradient is clipped: norms are 300 to 650 against $\gamma = 1$. g_F is the exact weighted gradient, g_J the one-sample gradient, C the clip.

	argmax student (seed 0)	pretrained init
mean pairwise cosine of the four candidate gradients	0.17 [0.14]	0.43 [0.43]
$\cos(g_{j^\star}, g_F)$, argmax candidate against the exact gradient	0.87 [0.92]	0.91 [0.94]
candidate-sampling variance $V_c/\|g_F\|^2$	0.99 [0.76]	0.51 [0.42]
Adam second-moment ratio $\sum_j \pi_j \|g_j\|^2/\|g_F\|^2$	1.99 [1.76]	1.51 [1.42]
clipping difference $B_c/\|C(g_F)\|$	0.27 [0.27]	0.17 [0.15]
$\cos\big(\mathbb{E}[C(g_J)],\, C(g_F)\big)$	0.990 [0.997]	0.995 [0.998]
$\|\mathbb{E}[C(g_J)]\|/\|C(g_F)\|$	0.75 [0.75]	0.85 [0.86]
same candidate, second Δ draw: $\|g - g'\|^2/\|g\|^2$	0.78 [0.70]	1.05 [0.59]
same candidate, second Δ draw: $\cos(g, g')$	0.67	0.65
batched against single rollout: max state difference, cos of gradients	0, 0.99999	0, 0.99999

of order one: its noise power equals the signal power, and the raw-gradient second-moment ratio is 2.0. Because clipping precedes AdamW, this bounds what Adam's second moment sees rather than measuring it. Under clipping the expected one-sample update is nearly parallel to the exact clipped update (cosine 0.99) but 25% shorter in expectation; how much of that shortening survives Adam's per-coordinate normalisation was not measured, so the sampled arm's steps are shorter in the clipped gradient, not necessarily by 25% in the parameters. And the Δ draw alone, which every arm resamples on every visit, produces gradient noise of the same order (relative squared difference 0.78 between two draws on the same trajectory), so candidate resampling roughly doubles a noise source that is already there rather than introducing a qualitatively new one. Batched and single rollouts give bit-identical states. The sampled estimator is therefore unbiased for the gradient of the weighted loss, and under clipping and AdamW it need not reproduce the exact estimator's optimisation; the measured differences are modest in direction and moderate in magnitude, which is consistent with the small residual gap after averaging and does not require contradictory targets to explain.

11.5 A decode-free latent scorer

The scorer of Section 5 needs a VAE decode and a DINOv2 pass per candidate. Both are cheap offline (Section 14), but a scorer that reads the terminal latent directly would remove them and could later be used inside a training loop. A latent scorer was therefore built and tested as a separate design decision from the selection rule. It replaces only the candidate side of the score: a projector P maps the terminal latent z_K to the DINOv2 patch-mean space, and the score is the cosine to the reference photograph's unchanged DINO embedding,

$$\hat{S}_j = \big\langle P(z_K^{(j)}),\, u^{\mathrm{pat}}(x^{\mathrm{ref}}) \big\rangle. \tag{16}$$

P is a five-stage convolutional network with global pooling and a two-layer head (6.8M parameters). It was trained on 22,000 captions of the 118k pool, none of them in the 3k pool, with the terminal latents of their four cached candidates re-rolled from the cached seeds (the recomputed RGB scores match the cache to 0.002), on a cosine loss to the candidate's DINO embedding plus a within-caption ranking term, $\mathrm{KL}\big(\mathrm{softmax}(S/\tau) \,\|\, \mathrm{softmax}(\hat{S}/\tau)\big)$ at $\tau = 0.04$, and validated on 2,000 further held-out captions. Training took five minutes on one GPU. The best epoch was chosen by selection regret on the validation captions, and the 3k pool's candidates were then scored once, offline, so the distillation arms below differ from the RGB arms only in the cache field they rank by.

Table 27 gives the offline ranking quality and Table 28 the distillation result. With the latent scorer, argmax selection is $+0.0103 \pm 0.0075$ CompBench over random after averaging

Table 27: The latent scorer on the 3k pool, whose captions and photographs it never saw. Regret is the RGB DINO score of the RGB argmax minus that of the latent argmax; headroom is measured against the VQAScore best-of-four oracle as in Table 4.

	latent scorer	RGB DINOv2 patches
top-1 agreement with the RGB argmax	0.486 (chance 0.25)	1
within-caption Spearman with the RGB score	0.47	1
selection regret (RGB-score units; top-2 margin is 0.049)	0.029	0
VQAScore of the pick (random 0.843, oracle 0.933)	0.874	0.882
VQAScore headroom recovered	34.0%	43.6%
top-1 agreement with the VQAScore oracle	0.32	0.36
decode + scorer cost per caption	0	61 + 32 ms

Table 28: Distillation with the latent scorer, 3k pool, three seeds, same configuration and trainer as Table 22. CompBench per seed and seed mean ± sd under both protocols; seed-paired contrasts after averaging.

	raw final		averaged		averaged, seed-paired	
Rule	s0 / s1 / s2	mean ± sd	s0 / s1 / s2	mean ± sd	vs random	vs the RGB rule
latent scorer, argmax	0.4528 / 0.4701 / 0.4700	0.4643 ± 0.0100	0.4773 / 0.4877 / 0.4871	0.4840 ± 0.0059	$+0.0103 \pm 0.0075$	-0.0033 ± 0.0047
latent scorer, Boltzmann T=0.04, exact	0.4732 / 0.4747 / 0.4572	0.4684 ± 0.0097	0.4818 / 0.4874 / 0.4731	0.4807 ± 0.0072	$+0.0070 \pm 0.0076$	-0.0032 ± 0.0045
RGB argmax (Table 22)	0.4690 / 0.4591 / 0.4825	0.4702 ± 0.0118	0.4849 / 0.4861 / 0.4909	0.4873 ± 0.0032	$+0.0135 \pm 0.0042$	–
RGB Boltzmann T=0.04, exact	0.4714 / 0.4709 / 0.4684	0.4703 ± 0.0016	0.4855 / 0.4859 / 0.4805	0.4840 ± 0.0030	$+0.0102 \pm 0.0031$	–

and 0.0033 ± 0.0047 below RGB argmax; exact Boltzmann weighting is $+0.0070 \pm 0.0076$ over random and 0.0032 ± 0.0045 below its RGB counterpart, and as before it is not better than argmax. Neither latent arm is distinguishable from its RGB counterpart at three seeds, and the point estimates sit at about three quarters of the RGB gain, which is the fraction of the RGB scorer's offline headroom the projector recovers. Fidelity follows the same pattern (Table 25): after averaging, CMMD is 0.81 for latent argmax against 0.80 for RGB argmax and 0.79 for both exact arms, with seed-paired differences of $+0.013 \pm 0.032$ and 0.000 ± 0.010, and precision and recall within 0.006. One property of the projector matters for any use beyond offline selection: it was fitted to rank frozen-teacher candidates, and on the students' own 4-step samples its agreement with the RGB scorer drops (top-1 0.37, Spearman 0.22, against 0.51 and 0.46 on teacher candidates; the students' samples are also half as spread across seeds, 0.042 against 0.078), and fine-tuning it on 1,000 student latents does not restore it. It is a scorer of the teacher's candidate distribution. A scorer that reads the terminal latent therefore carries most of the selection signal at no decode cost, with a projector that trains in minutes; how much of the remaining quarter a larger projector, more training captions or a multi-scale readout would recover was not tested, and the same reference photograph is still required, since only the candidate side of the score moved into latent space.

11.6 The projector as a reward inside training

A scorer that reads latents can be applied inside the training loop, where decoding every prediction would not be affordable. This was tested as a reward term on the student's own predictions, with the projector both frozen and updated as the student changes. Before training, the precondition was measured: on the students' own 4-step samples the frozen projector agrees with the RGB scorer much less than on teacher candidates (Section 11.5: top-1 0.37 against 0.51), and fine-tuning it offline on 1,000 student latents does not raise that, so the expectation on record was a small or null effect, to be judged by a hacking monitor rather than by the reward alone.

Design. At every update the reward $r = \langle P(\hat{x}_0), u^{\text{pat}}(x^{\text{ref}}) \rangle$ is computed on the two clean estimates $\hat{x}_0$ from the least-noisy student inputs (the estimates from the noisier inputs are too blurry to score) and $-\lambda r$ is added to the consistency loss of the argmax arm, so the gradient flows through the frozen projector weights into the student. λ was set from a probe of the two gradients on eight updates: at $\lambda = 1$ the reward gradient has 0.25% of the consistency gradient's norm (1.4 against 620), so $\lambda = 80$ puts it at 20%. Every 100 updates the 32 most recent predictions are decoded and scored with the RGB DINO scorer against their references, and the RGB score, the projector score and their correlation over the batch are logged; this is the monitor. In the *refreshed* arm the projector then takes four AdamW steps on those decoded predictions (cosine to their DINO embeddings) mixed with replay of 2,000 teacher-candidate captions under the original loss, so it tracks the RGB scorer on the student's prediction distribution; in the *frozen* arm it does not. Three seeds each, 3k configuration.

Table 29: Projector reward, 3k pool, three seeds. Left: the monitor at the start (updates 100 to 300) and the end (5,800 to 6,000) of training, means over seeds: DINO score of the decoded predictions under the projector (the reward) and under the RGB scorer, and their correlation over the 32 decoded latents. Right: CompBench and GenEval2 of the trained students, raw final and averaged checkpoints, with seed-paired contrasts against the argmax arm after averaging.

	projector score		true RGB score		correlation		CompBench	CompBench	vs argmax
Arm	start	end	start	end	start	end	raw	averaged	averaged
argmax (reference, Table 22)	–	–	–	–	–	–	0.4702 ± 0.0118	0.4873 ± 0.0032	–
argmax + reward, frozen projector	0.396	0.427	0.589	0.579	0.32	0.36	0.4758 ± 0.0068	0.4847 ± 0.0075	-0.0026 ± 0.0050
argmax + reward, refreshed projector	0.398	0.435	0.589	0.578	0.33	0.41	0.4773 ± 0.0058	0.4876 ± 0.0096	$+0.0003 \pm 0.0075$

Figure 12: The hacking monitor of the six reward runs. Left: projector score (solid) and true RGB score (dashed) of the same decoded predictions over training. Middle: their change since update 100. Right: correlation between the two over each monitor batch. The reward rises in every run; the RGB score does not follow.

Table 29 and Figure 12 give the result. Three things happened, in every seed of both arms. The reward went up: the projector score of the decoded predictions rose by 0.02 to 0.04, and the mean reward seen by the optimizer from 0.385 to 0.435 (frozen) and 0.447 (refreshed). The true score did not follow: the RGB DINO score of the same predictions moved by -0.024, -0.006, 0.000 (frozen) and -0.005, -0.026, -0.001 (refreshed) per seed, never up. And the benchmarks did not move: after averaging, the frozen-projector arm is 0.0026 ± 0.0050 below argmax and the refreshed arm 0.0003 ± 0.0075 above it on CompBench, with GenEval2 at 24.2 and 24.0 against 23.7, all within seed spread; the refreshed arm is 0.003 ± 0.003 above the frozen one, a consistent sign below resolution. The refresh did what it could: the correlation between projector and RGB scores over the monitor batches rose from 0.33 to 0.41 (against 0.36 for the frozen projector) but its fitting loss on the student's predictions stayed near 1.0 throughout, so four steps on 32

latents every 100 updates never caught up with the moving prediction distribution. Fidelity did not move either (Table 25): after averaging, the frozen-projector arm is at CMMD 0.81 and the refreshed arm at 0.84 against 0.80 for argmax, that is $+0.010 \pm 0.017$ and $+0.043 \pm 0.021$ ($p = 0.07$) worse, with precision and recall within 0.01 of argmax; on raw final checkpoints the differences are -0.040 ± 0.044 and $+0.013 \pm 0.055$, inside the seed spread. The refreshed arm is the slightly worse of the two on every seed after averaging ($+0.033 \pm 0.015$ CMMD, $p = 0.06$), consistent with a reward that pulls the predictions toward what the projector, not the RGB scorer, rewards. The pattern, a rising proxy with a flat or falling target and no benchmark change, is the signature of a student optimising the scorer rather than what the scorer measures. The projector is a scorer of the frozen teacher's candidates; as a reward on the student's own predictions, at a fifth of the gradient budget, it is inert on alignment and fidelity and is not adopted. Making it a usable reward would need a projector trained on student predictions at the supervised noise levels, with far more of them than a training loop yields cheaply, and a reward that is checked against the RGB score throughout; that is a separate study.

11.7 The exact reward: the RGB scorer inside the loop

Section 11.6 leaves one question open: whether the projector was the weak link, or whether a reward on the student's own predictions does nothing for the benchmarks however it is computed. The direct test replaces P with the RGB scorer itself, $\phi = u^{\text{pat}} \circ \text{Dec}$: each clean estimate is decoded by the VAE inside the training loop and scored by DINOv2 against the reference embedding, with the gradient flowing through the decoder and DINOv2 (both frozen) into the student. This is the reference experiment for the reward formulation, not an upper bound on rewards in general: it is one reward, one weight and one schedule, and a null here shelves this formulation only. Everything else is the argmax arm.

- **Differentiable path.** $\hat{x}_0$ is unscaled to the VAE's latent range, decoded (with activation checkpointing), mapped from $[-1, 1]$ to $[0, 1]$, resized to 256 pixels with an antialiased bicubic filter, centre-cropped to 224, normalised with the ImageNet statistics and embedded by DINOv2-B in fp32; the patch mean is L2-normalised and its cosine to $u^{\text{pat}}(x^{\text{ref}})$ is the reward. No `no_grad`, no 8-bit quantisation and no PIL step anywhere on the path.

- **Verification against the offline scorer.** On 16 cached teacher latents the differentiable score agrees with the offline PIL-based scorer that built the caches to a maximum difference of 0.011 (mean 0.0026, correlation 0.9998); the PIL path itself matches the cached scores to 0.0016. The PIL path is kept as the independent monitor: every 100 updates the 32 most recent predictions are decoded to 8-bit images and scored offline, exactly as in Section 11.6.

- **Weight from a gradient probe.** On eight updates at $\lambda = 1$ the reward gradient has median norm 7.1 against 497 for the consistency gradient (ratio 0.013), with cosine 0.04 between the two, so at the measured points the two gradients are nearly orthogonal. $\lambda = 15.5$ puts the raw reward gradient norm at 20% of the raw consistency gradient norm over those eight updates, the same rule the projector arms used. This sets an initial scale; it is not 20% of the AdamW update, which clips the sum to norm one and normalises per coordinate, and the ratio was not tracked through training in these runs (the factorial re-run of Section 11.8 logs it every 500 updates).

- **Cost.** The decode and DINOv2 pass add 30% to the step time in the probe (1.72 against 1.31 seconds) and 17% to the wall-clock of a full run (81 against 69 minutes), with the reference embeddings precomputed once for the pool.

- **Protocol.** Same 3k pool, argmax weights, three seeds, schedule and checkpoint averaging as the argmax arm; the averaged model is the primary comparison. Besides CompBench,

GenEval2 and fidelity, both arms and the random baseline are scored on 16 held-out validation captions (never in the pool) at every saved checkpoint: the RGB DINO score of the supervised clean estimates $\hat{x}_0$ at states 5 and 6 of the argmax teacher trajectory, and of complete 4-step samples (two seeds per caption), both by the offline scorer.

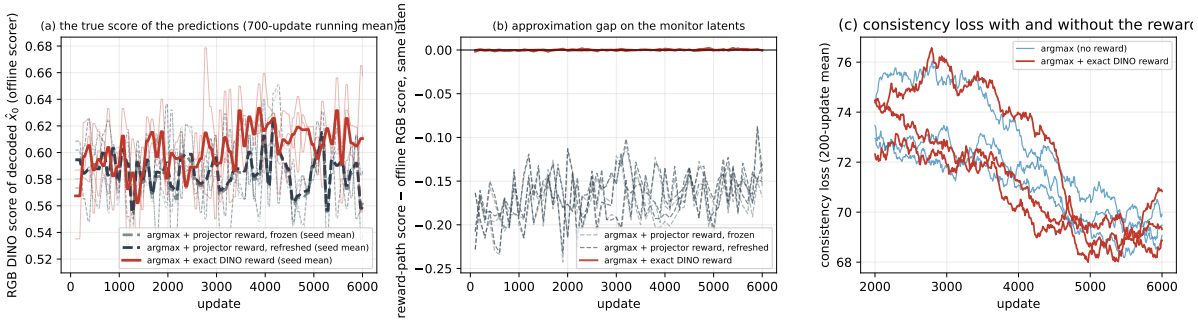

Figure 13: The exact reward against the projector rewards, in training. (a) The offline RGB DINO score of the decoded predictions (the hacking monitor, 32 latents every 100 updates, 700-update running mean; bold is the seed mean). (b) The approximation gap on matched latents: the score the reward path assigns to the 32 monitor latents minus their offline RGB score, one point per monitor evaluation. For the exact reward the gap is 0.000 ± 0.001 (largest 0.0025); for the projector arms it is -0.17, and it is the projector's error on student predictions, not a difference of sample population, noise level or time window. (c) The consistency loss of the exact-reward arm against argmax without a reward, 200-update means.

In training. Figure 13 shows what the projector arms could not do. The true RGB score of the predictions rises: from 0.583 over updates 100 to 300 to 0.608 over the last 200 updates, per seed +0.018, +0.050 and +0.006, where the projector arms fell by 0.014 (Table 29). On the monitor latents the differentiable path and the offline scorer agree to 0.0025 (mean gap 0.000, standard deviation 0.0007 over the 60 monitor evaluations of each run, correlation 0.9997), so there is no learned latent surrogate to exploit. DINO similarity itself remains a proxy for what the benchmarks and a viewer judge; the agreement of two implementations of the same scorer verifies implementation consistency, not alignment, and the benchmark, held-out and fidelity numbers below are the evidence on that. The measured consistency loss changes little (70.5 against 69.6 for argmax over the last 1,000 updates), so the reward is not visibly paid for with the distillation objective. The reward is optimised for real. What that buys on held-out captions and on the benchmarks is the question.

Table 30: The reward arms on the benchmarks, 3k pool, three seeds, seed mean ± standard deviation over seeds: raw final checkpoints and the average of the 2k/4k/6k checkpoints, with the seed-paired contrast of each arm against argmax after averaging (mean ± sd of the per-seed differences, seed t-test p). The exact-reward rows are positive against argmax on every seed, both benchmarks, both protocols.

	raw final		averaged		vs argmax, averaged	
Arm	CompBench	GenEval2	CompBench	GenEval2	CompBench	GenEval2
argmax (reference)	0.4702 ± 0.0118	22.46 ± 1.36	0.4873 ± 0.0032	23.73 ± 1.55	–	–
argmax + projector reward, frozen	0.4758 ± 0.0068	22.93 ± 0.44	0.4847 ± 0.0075	24.19 ± 0.36	-0.0026 ± 0.0050 ($p = 0.47$)	$+0.46 \pm 1.57$ ($p = 0.66$)
argmax + projector reward, refreshed	0.4773 ± 0.0058	22.40 ± 1.22	0.4876 ± 0.0096	23.97 ± 0.34	$+0.0003 \pm 0.0075$ ($p = 0.95$)	$+0.24 \pm 1.22$ ($p = 0.76$)
argmax + exact DINO reward	0.4866 ± 0.0083	25.21 ± 2.18	0.4984 ± 0.0038	25.25 ± 0.31	$+0.0111 \pm 0.0060$ ($p = 0.085$)	$+1.52 \pm 1.25$ ($p = 0.17$)

Table 31: Held-out monitor: RGB DINO similarity to the reference photograph on validation captions never in the pool, offline scorer, seed mean ± sd over three seeds; 16 captions at every checkpoint (top), and a separate draw of 300 captions on the averaged models (bottom). Left: the student's clean estimates $\hat{x}_0$ from the two least-noisy supervised inputs on the argmax teacher trajectory (the quantity the reward acts on). Right: complete 4-step samples at $w = 1$, two seeds per caption. The teacher's own argmax candidate scores 0.528 on these captions. Contrasts are seed-paired (mean ± sd of per-seed differences, seed t-test p).

Arm	checkpoint	$\hat{x}_0$ DINO	4-step sample DINO
random (fixed draw)	2k / 4k / final / averaged	0.5267 / 0.5311 / 0.5293 / 0.5314	0.4765 / 0.4821 / 0.4836 / 0.4900
argmax	2k / 4k / final / averaged	0.5287 / 0.5288 / 0.5292 / 0.5298	0.4836 / 0.4949 / 0.4918 / 0.5052
argmax + exact DINO reward	2k / 4k / final / averaged	0.5350 / 0.5383 / 0.5380 / 0.5379	0.4939 / 0.5131 / 0.5048 / 0.5117
argmax − random	averaged	-0.0015 ± 0.0010 ($p = 0.11$)	$+0.0152 \pm 0.0183$ ($p = 0.29$)
exact reward − argmax	2k	$+0.0063 \pm 0.0014$ ($p = 0.02$)	$+0.0103 \pm 0.0155$ ($p = 0.37$)
exact reward − argmax	4k	$+0.0095 \pm 0.0011$ ($p = 0.004$)	$+0.0181 \pm 0.0265$ ($p = 0.36$)
exact reward − argmax	averaged	$+0.0080 \pm 0.0011$ ($p = 0.01$)	$+0.0065 \pm 0.0083$ ($p = 0.31$)
exact reward − random	averaged	$+0.0065 \pm 0.0009$ ($p = 0.01$)	$+0.0217 \pm 0.0187$ ($p = 0.18$)
300 validation captions (a separate draw), averaged models, three seeds			
random (fixed draw)	averaged	0.5970 ± 0.0005	0.5831 ± 0.0029
argmax	averaged	0.5973 ± 0.0006	0.5888 ± 0.0060
argmax + exact DINO reward	averaged	0.6094 ± 0.0002	0.6080 ± 0.0012
argmax − random	averaged	$+0.0003 \pm 0.0011$ ($p = 0.72$)	$+0.0056 \pm 0.0051$ ($p = 0.19$; caption-level $p = 6 \times 10^{-5}$)
exact reward − argmax	averaged	$+0.0121 \pm 0.0005$ ($p = 0.001$; wins 83% of captions)	$+0.0192 \pm 0.0065$ ($p = 0.04$; caption-level $p = 6 \times 10^{-37}$, wins 69%)
exact reward − random	averaged	$+0.0124 \pm 0.0006$ ($p = 0.001$)	$+0.0248 \pm 0.0023$ ($p = 0.003$; wins 72%)

Benchmarks. Table 30. After identical averaging the exact-reward students score 0.4984 on CompBench against 0.4873 for argmax, $+0.0111 \pm 0.0060$ with every seed positive (+0.0119, +0.0166, +0.0047; $p = 0.085$ at $n = 3$), and $+0.0246 \pm 0.0049$ over the random baseline ($p = 0.013$): the reward roughly doubles what selection alone gives (+0.0135). GenEval2 moves from 23.7 to 25.3, $+1.5 \pm 1.3$, again positive on every seed, and $+1.3 \pm 0.2$ ($p = 0.005$) over the refreshed-projector arm at the same reward budget. On raw final checkpoints the gaps are larger and noisier ($+0.016 \pm 0.017$ CompBench, $+2.8 \pm 0.9$ GenEval2, $p = 0.03$), and the exact-reward arm's seed spread after averaging is small (0.0038 CompBench, second only to argmax's 0.0032; 0.31 GenEval2, the smallest of any arm). Under the official ten-images-per-prompt CompBench protocol on the same averaged models, the exact-reward students score 0.4987 against 0.4904 for argmax and 0.4774 for random pick (per seed +0.0125, +0.0097, +0.0024 over argmax, $+0.0082 \pm 0.0052$; $+0.0212 \pm 0.0030$ over random, $p = 0.007$), so the one-image protocol did not flatter the reward. One qualification on the pairing: loading the scorer inside the trainer advances the global generator before the first data draw, so the reward arms see a different caption order from the argmax arm at the same seed. The contrasts above pair runs that share initial weights, noise-level draws and evaluation prompts but not the caption order; that makes them conservative (the pairing removes less variance), not biased, and an unpaired Welch test on the same six averaged CompBench values gives $p = 0.02$.

Held-out captions. Table 31. On captions the students never trained on, the reward does what it was asked: the DINO similarity of the supervised clean estimates rises by $+0.0080 \pm 0.0011$ over argmax ($p = 0.01$), at every checkpoint from 2k on, and past the teacher's own argmax candidate (0.538 against 0.528), so the student's estimates are closer to the photographs than the trajectories it distils. On complete 4-step samples the 16-caption set gives $+0.0065 \pm 0.0083$ over argmax (caption-level $p = 0.22$) and +0.0217 over random (caption-level $p = 0.001$), which is not resolved at three seeds. A separate draw of 300 validation captions on the averaged models resolves it (Table 31, bottom): the clean estimates gain $+0.0121 \pm 0.0005$ over argmax ($p = 0.001$, the reward student ahead on 83% of captions), and the deployed 4-step samples gain $+0.0192 \pm 0.0065$ ($p = 0.04$ over seeds, caption-level $p = 6 \times 10^{-37}$, ahead on 69% of captions) and $+0.0248 \pm 0.0023$ over random. Selection alone moves the samples by +0.0056 on these captions and the clean estimates not at all, so on the quantity the reward optimises, the reward is worth three to four times the selection, and it carries through to the samples the student actually produces.

Fidelity. Table 25. This is the largest fidelity change of any arm in the study. After averaging, CMMD falls from 0.80 (argmax) to 0.69, -0.11 ± 0.04 with every seed improving (-0.15, -0.11, -0.07), precision rises from 0.489 to 0.541 ($+0.051$ on every seed) and recall from 0.066 to 0.088 ($+0.021$); FID is level (30.4 against 30.1, inside its split-half spread). On raw checkpoints CMMD is 0.77 against 0.89. The direction is what the reward asks for: it measures similarity to real photographs, so pulling the predictions toward it pulls the sample distribution toward COCO. The projector arms, whose reward the true score did not follow, moved CMMD the other way ($+0.01$ and $+0.04$).

What this establishes. The reward formulation was not the weak link; the projector was. With the exact score in the loop, the same $-\lambda r$ term at the same 20% gradient budget improves alignment ($+0.011$ CompBench, $+1.5$ GenEval2), the held-out quantity it optimises ($+0.008$), and fidelity (-0.11 CMMD, $+0.05$ precision) at once, for 17% more training time and no change at inference. These first three runs saw a different caption order from the argmax arm; the factorial of Section 11.8, five seeds per cell with the order matched, puts the reward's own contribution at $+0.0055 \pm 0.0021$ CompBench ($p = 0.004$) and its fidelity gain at -0.07 CMMD, additive with selection. The held-out gains, on clean estimates and on deployed samples alike, are resolved on 300 captions. The reward reads the same reference photograph the scorer reads, so the method now uses the photograph twice, once to choose the trajectory and once to pull the predictions toward it, and the second use is worth about as much as the first. It survives a λ sweep (Section 11.9, $\lambda \approx 31$ combined with rewarding all five states is the best cell found) but does not add cleanly to a stronger teacher (Section 12.4, gain in the same direction but not resolved at three seeds); it does hold at the 118k scale (Section 10.7: $+0.0087$ CompBench and -0.10 CMMD over argmax alone, every seed). The release code runs the arm with `--reward_mode rgb`.

11.8 The selection × reward factorial

The exact-reward result is one cell of a two-by-two design that the study so far has three cells of. Write M_{sr} for the averaged CompBench of the arm with selection $s \in \{0, 1\}$ (random pick or argmax) and reward $r \in \{0, 1\}$. $M_{00} = 0.4738$, $M_{10} = 0.4873$ and $M_{11} = 0.4984$ are known; M_{01}, random trajectories with the exact reward, is not. Without it three readings of Section 11.7 cannot be told apart: selection and reward are complementary; the reward supplies most of the useful supervision and selection is largely redundant; or selection matters because it puts the student on trajectories where the reward gradient is useful. The practical quantity is the selection benefit once the reward is present, $M_{11} - M_{01}$, and the interaction is $I = (M_{11} - M_{10}) - (M_{01} - M_{00})$. A positive interaction is not needed to justify using both; the missing cell is needed to know whether both are necessary, and if random trajectories with the reward match selected ones, the four-candidate pool and its scoring could be dropped from the pipeline.

The design was predeclared before any cell was run.

- **Cells and seeds.** Five training seeds per cell. The two reward-free cells keep their seeds 0 to 2 and add seeds 3 and 4 under the same frozen trainer; the two reward cells were trained fresh for seeds 0 to 4 under the RNG-restored trainer below, so the first exact-reward runs of Section 11.7 become a replication under a different caption order rather than a cell of the design.
- **Recipe fixed.** The reward is exactly that of Section 11.7: $\phi = u^{\text{pat}} \circ \text{Dec}$, $\lambda = 15.5$, two states, monitor every 100 updates. Nothing else changes between cells but the selection rule and the presence of the reward. The raw gradient-norm ratio and cosine are logged every 500 updates so the 20% rule can be checked through training rather than at the first eight updates.

- **Independent RNG streams.** The trainer saves the global generator state before loading the reward machinery and restores it after, so a reward arm visits the captions in the same order as the reward-free arm at the same seed. The gate for launching was a zero-weight control that the previous code could not run: a reward mode with $\lambda = 0$ loads the scorer but adds no term, and it was bit-identical to the reward-free run on all 909 parameter tensors after six updates, with the reward run's per-update consistency loss matching the reward-free run's at every update.

- **Analysis.** Seed-paired contrasts within each column and row of the table, the interaction I with its seed-level interval, an unpaired two-way least-squares fit over all twenty runs as a check, and the same fidelity measurement as before; the primary comparison is on averaged checkpoints.

Table 32: The selection × reward factorial on the 3k pool, averaged checkpoints, five seeds per cell (seed mean ± sd). Effects are seed-paired differences (mean ± sd over seeds, seed t-test p); the interaction is the per-seed $(M_{11} - M_{10}) - (M_{01} - M_{00})$. CMMD on 5,000 COCO captions, lower is better.

	CompBench			GenEval2 (×100)		
	no reward	exact reward	reward effect	no reward	exact reward	reward effect
random pick	0.4751 ± 0.0032	0.4819 ± 0.0060	$+0.0068 \pm 0.0060$ ($p = 0.07$)	22.47 ± 0.68	23.06 ± 0.65	$+0.60 \pm 0.89$
argmax	0.4868 ± 0.0031	0.4923 ± 0.0021	$+0.0055 \pm 0.0021$ ($p = 0.004$)	23.92 ± 1.29	24.41 ± 0.77	$+0.49 \pm 1.30$
selection effect	$+0.0117 \pm 0.0058$ ($p = 0.011$)	$+0.0104 \pm 0.0062$ ($p = 0.019$)	$I = -0.0013 \pm 0.0060$ ($p = 0.66$)	$+1.46 \pm 1.38$	$+1.35 \pm 1.14$	$I = -0.11 \pm 1.57$
	CMMD ↓				precision / recall	
random pick	0.846 ± 0.035	0.750 ± 0.024	-0.096 ± 0.025 ($p = 0.001$)	0.478 / 0.055	0.503 / 0.074	
argmax	0.794 ± 0.023	0.726 ± 0.017	-0.068 ± 0.018 ($p = 0.001$)	0.497 / 0.066	0.522 / 0.082	+0.025 / +0.016
selection effect	-0.052 ± 0.057 ($p = 0.11$)	-0.024 ± 0.035 ($p = 0.20$)				

Result. Table 32. The two effects are additive: the interaction is -0.0013 ± 0.0060 on CompBench and -0.1 ± 1.6 on GenEval2, zero within noise on both, and the unpaired two-way fit over all twenty runs agrees (selection 0.0117 ± 0.0025, reward 0.0068 ± 0.0025, interaction -0.0013 ± 0.0035). Selection stays necessary once the reward is present: $M_{11} - M_{01} = +0.0104 \pm 0.0062$ ($p = 0.019$), the same size as without it, so random trajectories with the reward do not reach argmax alone and the four-candidate pool cannot be dropped. The reward's own contribution, with the caption order matched across arms, is $+0.0055 \pm 0.0021$ CompBench over argmax ($p = 0.004$) and $+0.0068 \pm 0.0060$ over random pick; on GenEval2 it is +0.5 to +0.6 and unresolved at five seeds. The three readings of Section 11.7 therefore resolve to the first: selection and reward are complementary, each worth about what it is worth alone, with selection the larger of the two on alignment. On fidelity the order reverses: the reward lowers CMMD by 0.07 to 0.10 with $p = 0.001$ in both rows and raises precision and recall, while selection's fidelity effect is half that size and not resolved.

The caption-order effect, measured. The first exact-reward runs of Section 11.7 scored 0.4984; the same recipe under the matched caption order scores 0.4923, a difference of -0.0055 ± 0.0033 on the three shared seeds ($p = 0.10$) and -1.2 ± 0.7 GenEval2. Half of the +0.011 first reported as the reward's gain over argmax was therefore the caption order those runs happened to see, which is exactly the confound the factorial was built to remove. The first-round numbers are kept in Tables 30 and 25 as a replication under a different order, not as the estimate.

Against naive distillation. The practical comparison is the diagonal of the table: argmax selection with the exact reward against random pick without it, the plain recipe. After averaging it is $+0.0172 \pm 0.0051$ CompBench ($p = 0.002$) and $+1.94 \pm 0.68$ GenEval2 ($p = 0.003$) over five seeds, with CMMD 0.73 against 0.85; every CompBench category moves in the same direction under both the one-image and the official ten-image protocol, and on GenEval2 every skill

and every complexity bucket gains, with fewer collapsed prompts (31% against 37%). The per-category and per-skill tables are in the release documentation.

Two further experiments followed the factorial: a one-factor ablation of the recipe (Section 11.9), and the reward with the 16-step and 28-step teachers at fixed selection (Section 12.4), which tests whether the reward contributes beyond filtering the weaknesses of the 8-step teacher, where the selection gain itself disappears.

11.9 One-factor ablations of the exact-reward recipe

The recipe has four settings that were chosen once and never varied: the weight $\lambda = 15.5$ from the 20% gradient rule, the reward on the two least-noisy of the five supervised clean estimates, an antialiased bicubic resize on the differentiable path, and DINOv2 in fp32. Each was changed on its own against the factorial's argmax + exact reward arm (the recipe, RNG-restored trainer, 0.4923 CompBench), three seeds per arm, everything else fixed, plus two arms that push the projector reward of Section 11.6 harder (four times more refresh steps; refreshes four times as often) to close that line. Table 33 and Figure 14 give the results on the averaged checkpoints; the table also reports the in-training monitor of each arm (the change of the true RGB score of the predictions from the first to the last 200 updates) and the logged raw gradient-norm ratio averaged over training.

Table 33: One-factor ablations of the exact-reward recipe, 3k pool, averaged checkpoints, three seeds per arm (the recipe and argmax rows are the factorial's five-seed cells). Contrasts are seed-paired against the recipe (mean ± sd, seed t-test p); the projector rows are contrasted against argmax, which they do not beat. Monitor: change of the offline RGB DINO score of the decoded predictions between updates 100–300 and 5,800–6,000; ratio: $\|\nabla(\lambda r)\|/\|\nabla L_{\mathrm{CD}}\|$ averaged over the probes every 500 updates.

Arm	change	CompBench	vs recipe	GenEval2	CMMD ↓	prec. / recall	FID	monitor / ratio
argmax, no reward	reference	0.4868	−0.0055	23.92	0.79	0.497 / 0.066	30.0	–
recipe	$\lambda = 15.5$, 2 least-noisy states	0.4923	–	24.41	0.73	0.522 / 0.082	29.9	+0.012 / 0.30
$\lambda = 7.75$	half the weight	0.4910	-0.0020 ± 0.0035 ($p = 0.43$)	25.01	0.76	0.498 / 0.075	30.0	+0.005 / 0.14
$\lambda = 31$	twice the weight	0.4989	$+0.0060 \pm 0.0050$ ($p = 0.18$)	25.51	0.69	0.534 / 0.091	30.6	+0.021 / 0.49
$\lambda = 62$	four times the weight	0.5034	$+0.0105 \pm 0.0025$ ($p = 0.02$)	24.89	0.63	0.561 / 0.101	31.4	+0.032 / 0.88
$R = 1$	least-noisy state only	0.4933	$+0.0004 \pm 0.0031$ ($p = 0.84$)	24.36	0.75	0.502 / 0.082	30.1	+0.004 / 0.25
$R = 5$	all five states	0.5022	$+0.0093 \pm 0.0025$ ($p = 0.02$)	25.27	0.69	0.523 / 0.089	30.8	−0.002 / 0.34
two noisiest states	instead of the two least-noisy	0.5037	$+0.0108 \pm 0.0009$ ($p = 0.002$)	23.08	0.70	0.531 / 0.086	31.5	+0.205 / 0.50
bilinear resize	no antialiasing	0.4940	$+0.0011 \pm 0.0039$ ($p = 0.68$)	23.92	0.72	0.516 / 0.086	30.2	+0.002 / 0.30
bf16 DINO	half precision in the loop	0.4961	$+0.0031 \pm 0.0011$ ($p = 0.04$)	24.11	0.72	0.521 / 0.082	29.9	+0.001 / 0.30
projector, 16 refresh steps	vs argmax	0.4887	$+0.0014 \pm 0.0035$ ($p = 0.57$)	23.54	0.82	0.474 / 0.067	30.9	−0.019 / –
projector, refresh every 25	vs argmax	0.4866	-0.0007 ± 0.0008 ($p = 0.27$)	23.38	0.83	0.473 / 0.063	30.9	0.000 / –
combined, $\lambda = 31$, $R = 5$	both changes at once	0.5082	$+0.0153 \pm 0.0017$ ($p = 0.012$)	23.59	0.64	0.547 / 0.099	31.5	– / 0.59

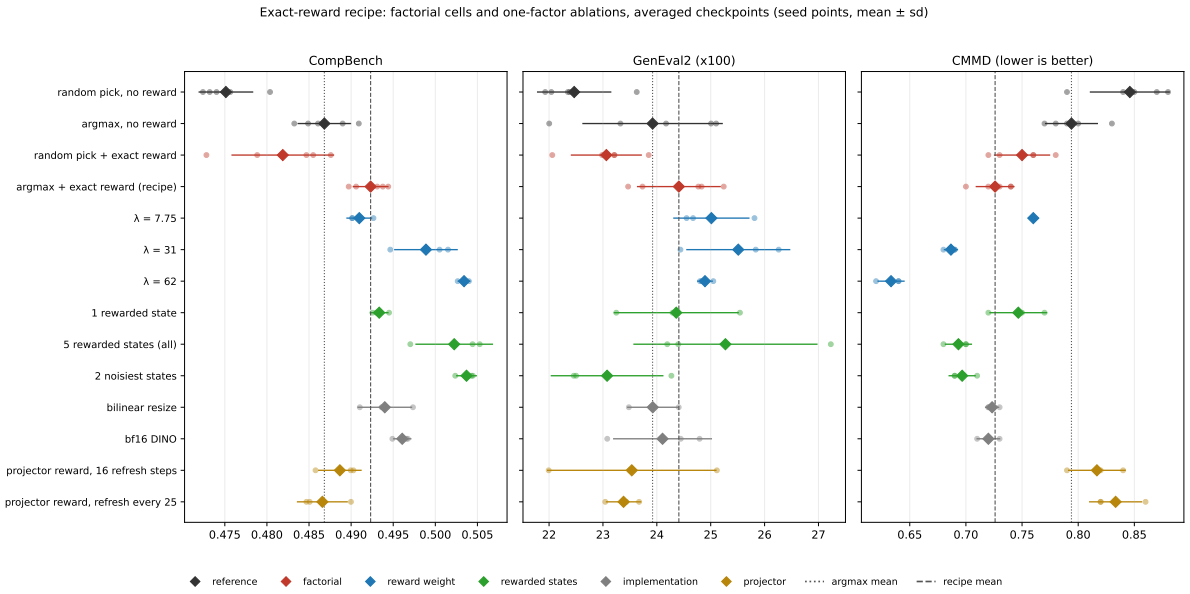

Figure 14: The factorial cells and the one-factor ablations on CompBench, GenEval2 and CMMD, averaged checkpoints: seed points, mean (diamond) and ± one seed standard deviation. Dotted line: argmax mean; dashed line: the recipe's mean.

- **The weight is the lever, and 15.5 is on the low side.** CompBench, CMMD, precision and recall all improve monotonically from $\lambda = 7.75$ to 62: +0.0105 CompBench ($p = 0.02$) and −0.09 CMMD ($p = 0.03$) over the recipe at $\lambda = 62$, with precision 0.561 and recall 0.101, the best fidelity of any arm in the study. The true score of the predictions rises with λ as the monitor says it should (+0.005, +0.012, +0.021, +0.032), the gradient-norm ratio averaged over training rises from 0.14 to 0.88, and the consistency loss at the end of training is 69.8, 70.5, 70.6 and 71.7, so the distillation objective begins to pay for the reward only at the top of the range. Two metrics dissent at $\lambda = 62$: FID rises by 1.5 while CMMD falls by 0.09 (the two disagree on what a 5,000-image set looks like when its images move toward the photographs), and GenEval2 peaks at $\lambda = 31$ (25.5) and drops back to 24.9 at 62. The 20% gradient rule therefore set a safe weight, not the best one; $\lambda \approx 31$ is the point where every measurement still agrees.

- **More rewarded states help; which states matters.** Rewarding all five clean estimates instead of the two least-noisy adds +0.0093 CompBench ($p = 0.02$), +0.9 GenEval2 and −0.03 CMMD at 1.3 times the recipe's step time, while rewarding only the least-noisy one changes nothing on CompBench and is +0.02 CMMD worse. Rewarding the two *noisiest* estimates instead of the two least-noisy is the strongest single change on CompBench ($+0.0108 \pm 0.0009$, $p = 0.002$) and equal to $R = 5$ on CMMD, but it costs 1.3 GenEval2. Its monitor explains the difference: the noisiest estimates start far from the photographs (true score 0.38 against 0.59) and the reward drags them up by 0.2, so this arm shapes what the student predicts from heavily noised inputs, the inputs that matter most for a 4-step sampler, at the price of the count-and-attribute precision GenEval2 scores. The assumption in Section 11.7 that the noisier estimates are too blurry to score was wrong: they are exactly the ones with the most to gain.

- **The implementation details do not matter.** Plain bilinear resizing and bf16 DINOv2 are within 0.003 CompBench and 0.003 CMMD of the recipe; the bf16 arm's +0.003 CompBench is at $p = 0.04$ but of the size of the seed spread. bf16 DINOv2 shortens the reward's forward and backward pass and is the cheaper default.

- **The projector reward is closed.** Four times more refresh steps or four times more frequent refreshes leave it level with argmax on CompBench ($+0.001$ and -0.001), below argmax on GenEval2, and worse on CMMD (0.82 and 0.83 against 0.79); the true score of the predictions still does not rise. The surrogate is the problem, not its training budget.

- **The two changes combine, on CompBench, past what either gives alone.** $\lambda = 31$ with all five states rewarded (three seeds) scores 0.5082 CompBench, $+0.0153 \pm 0.0017$ over the recipe ($p = 0.012$), $+0.0093 \pm 0.0013$ over $\lambda = 31$ alone ($p = 0.018$) and $+0.0060 \pm 0.0031$ over $R = 5$ alone ($p = 0.19$, not significant on its own but consistent in sign): the two components' gains are not simply redundant. CMMD falls to 0.64, matching the $\lambda = 62$ arm's fidelity without $\lambda = 62$'s GenEval2 cost, and precision rises to 0.547, the best of any reward arm. GenEval2 does not combine the same way: 23.59 is 1.9 points below $\lambda = 31$ alone ($p = 0.048$) and level with the recipe, so the CompBench and CMMD gains are bought at GenEval2's expense past the point either factor reaches alone. FID also moves the wrong way relative to either component alone (31.5 against 30.6 and 30.8), the same CMMD/FID disagreement already seen at $\lambda = 62$. Against naive selection (same three seeds) the combined recipe is $+0.0345$ CompBench ($p = 0.001$) and CMMD $0.84 \to 0.64$.

- **What is not established.** None of the ablation rows, including the combined recipe, has held-out or official-protocol measurements, and the combined recipe was only run at the 3k scale before the 118k scale up decision below.

11.10 What the evidence supports

The practical choice is settled: argmax selection, stored once, is the best rule tested and the cheapest. Exact soft weighting at $T \leq 0.08$ is not detectably better and costs about three times the wall-clock; softer rules and uniform weighting are worse or level with random. The explanation of why per-visit resampling underperforms is only partly settled. Its raw-checkpoint deficit is largely final-iterate noise, which checkpoint averaging removes; the remainder is of the size the clipping and second-moment effects measured above can produce, and the fixed-versus-resampled controls do not isolate a persistence effect from optimization variance and candidate coverage in this short-training regime. Separating them would take the same-caption Monte-Carlo estimator (which halves or quarters the sampling variance at single-caption compute) and more seeds; neither was run because the practical conclusion does not depend on it. A decode-free scorer that ranks candidates in latent space is a separate design decision from the selection rule, and Section 11.5 measures it: a small projector on the terminal latent recovers about three quarters of the RGB scorer's selection value offline and in trained students, at no decode cost; since decoding and scoring are under 10% of the offline selection pipeline (Section 14), its value would be as a scorer inside a training loop rather than as a saving, and Section 11.6 shows that, used as a reward on the student's own predictions, it is optimised without the true score or the benchmarks following. Section 11.7 then shows that the reward formulation itself is sound: with the exact RGB score decoded inside the loop, the same term at the same gradient budget raises CompBench by 0.011 and GenEval2 by 1.5 over argmax, lowers CMMD from 0.80 to 0.69 and raises the held-out DINO similarity of the predictions, on every seed, for 17% more training time. That is the one addition in this study that improves on argmax selection. The factorial (Section 11.8) shows it additive with selection and worth $+0.0055$ CompBench and -0.07 CMMD on its own, and the one-factor ablation (Section 11.9) shows the recipe under-weighted, with $\lambda \approx 31$ and all five states rewarded the settings to combine. Combining them (Section 11.9) is worth $+0.0153$ CompBench over the recipe and matches $\lambda = 62$'s CMMD without its GenEval2 cost, but past the point either factor reaches alone the CompBench and CMMD gain trades off against GenEval2 and FID rather than adding cleanly on every metric.

12 Ablations

All ablations use the 3k pool, three seeds and the 3k configuration unless stated.

12.1 Target horizon

The 1-step Tweedie target is compared with two targets that are closer to the trajectory endpoint: a 2-step chord, which uses z_{k+2} instead of z_{k+1}, and the endpoint z_8 itself. The 1-step target is the best for both arms and gives the largest selection gain (Table 34). The 1-step target is 27% off the endpoint at $k = 3$ and 10% at $k = 5$, so it is biased, but its lower variance across visits matters more.

Table 34: Target horizon. CompBench / GenEval2, seed means, and the selection gain of DINOv2 patches over random.

Target	random	DINOv2 patches	Selection gain (CompBench)
1-step Tweedie (used)	0.4584 / 20.73	0.4725 / 22.89	+0.0140
2-step chord	0.4533 / 21.09	0.4646 / 21.99	+0.0113 ($p = 3 \times 10^{-6}$)
endpoint z_8	0.4530 / 20.57	0.4565 / 20.97	+0.0035 (n.s.)

12.2 Inference step count

The students are trained for four steps and every number in this document samples them at four. Table 35 runs the averaged seed-0 argmax and exact-reward students at 2, 4, 8, 16 and 28 Euler steps at $w = 1$. Two steps collapse. Eight steps, which is the training grid itself, gives the highest CompBench for both students, but GenEval2 already falls, and from 16 steps on both benchmarks drift down rather than toward the 28-step teacher: the student learned to jump to the clean estimate from the states of the 8-step grid, not to take accurate small steps at noise levels it never saw. The exact reward's advantage is specific to the four-step budget it was trained for; at eight steps and above the argmax student is level or slightly ahead. The 4-step student is a 4-step model, and more inference compute should be spent on a stronger teacher, not on more student steps.

Table 35: Sampling the 4-step students with more steps: CompBench / GenEval2 (×100), averaged seed-0 models, one image per prompt, $w = 1$. The base model at 28 steps with $w = 7$ scores 0.5053 / 17.05.

Steps	argmax	argmax + exact reward
2	0.1798 / 2.25	0.1794 / 1.90
4 (trained for)	0.4849 / 24.17	0.4906 / 24.83
8	0.4996 / 22.76	0.4954 / 22.66
16	0.4932 / 20.99	0.4891 / 20.42
28	0.4908 / 19.80	0.4841 / 19.30

12.3 Objective: x_0 regression against segment-velocity imitation

A progressive-distillation-style objective places the student on state z_k and regresses the teacher's velocity chord to z_{k+1} with a mean-squared error. On the 8-step grid, deployed at 4 steps, it is much worse than the x_0 objective: random 0.4046 / 17.27 and VQAScore 0.4257 / 20.05 against 0.4584 / 20.73 and 0.4817 / 23.06. The selection gain persists under it (+0.021 CompBench), so selection does not depend on the objective, but the objective decides the absolute level.

12.4 Teacher quality

Two stronger teachers were tested with the same captions and seeds: 28 steps at $w = 7$ (candidates and training on the 28-step grid, supervised states 10, 14, 18, 22, 26, $\Delta \in \{3, \ldots, 10\}$), and 16 steps at the model-card guidance $w = 4.5$ (the 4-step inference grid nests in the 16-step grid; supervised states 6, 8, 10, 12, 14, $\Delta \in \{2, \ldots, 6\}$). Both compress the candidate pool a scorer can act on (Table 21): the oracle-minus-random VQAScore headroom falls from 0.0905 to 0.0415 (28 steps) and 0.0459 (16 steps), and the fraction of it the DINOv2 patch scorer recovers falls from 43.6% to 11.0% at 16 steps (top-1 agreement with the oracle 0.285, chance 0.25). The trained selection gain disappears (Table 36). A stronger teacher on its own is worth about as much on CompBench as scored selection on the 8-step teacher, and the two do not add. Guidance matters separately from step count: 16 steps at $w = 4.5$ lifts GenEval2 by 1.3 but CompBench by only 0.004 over the 8-step $w = 7$ teacher, while 28 steps at $w = 7$ lifts CompBench by 0.021.

Table 36: Teacher quality. CompBench / GenEval2, seed means over three seeds, and the paired selection gain within the same teacher.

Teacher	DINO headroom recovered	random	DINOv2 patches	Selection gain
8 steps, $w = 7$	43.6%	0.4584 / 20.73	0.4725 / 22.89	+0.0140 / +2.17
16 steps, $w = 4.5$	11.0%	0.4624 / 22.02	0.4664 / 22.16	+0.0040 ($p = 0.07$) / +0.14 ($p = 0.81$)
28 steps, $w = 7$	–	0.4797 / 23.06	0.4801 / 22.25	+0.0004 ($p = 0.87$) / −0.81 ($p = 0.18$)

Per teacher forward pass this is not a cost win for selection either: four 8-step candidates cost 32 passes per caption, one 28-step sample costs 28.

The exact-reward recipe (Section 11.7) was re-tried on both stronger teachers, three seeds each, to see whether it fares like scored selection or like the frozen-teacher exact reward. Raw checkpoints show a CompBench gain in the same direction as the 8-step teacher but not statistically resolved at three seeds: $+0.0051 \pm 0.0060$ on the 28-step teacher ($p = 0.48$) and $+0.0061 \pm 0.0054$ on the 16-step teacher ($p = 0.37$), against +0.021 (28 steps, argmax selection alone, Table 36) and +0.004 (16 steps) for selection without reward. GenEval2 does not move (−0.12 and +0.40, both n.s.). The averaged-checkpoint CompBench is 0.5023 (28-step teacher) and 0.4842 (16-step teacher), and averaged-checkpoint fidelity is CMMD 0.70 / FID 26.7 (28 steps) and CMMD 0.73 / FID 25.9 (16 steps) – both worse CMMD than the 8-step exact-reward recipe's 0.73 (Table 33), so a stronger teacher does not make the reward's fidelity gain larger either. The reward's benefit, like selection's, is largest on the teacher it was designed against; neither compensates for a stronger teacher's compressed candidate spread (Table 21), because the reward acts on the student's own predictions rather than on the teacher's candidates and so does not directly inherit that compression, but its gradient still has less true-score headroom to climb on a teacher that already samples closer to the photographs.

12.5 Regression onto the reference photograph

The reference photograph is the one sample we have from the distribution every scorer points at, so it was tested as a second regression target. On each update the photograph's latent is noised along its own straight path to two random supervised states, and the student's clean-latent estimate there is regressed onto the photograph latent with the same pseudo-Huber loss, added with weight $\lambda = 0.2$. On the 16-step teacher, three seeds, this term hurts both arms on both metrics: random 0.4624 → 0.4281 CompBench (-0.034, $p = 10^{-43}$, worst on UniDet at −0.052) and 22.02 → 20.67 GenEval2; DINOv2 patches 0.4664 → 0.4407 (−0.022) and 22.16 → 20.32. A single photograph per caption is a far noisier target than the teacher's trajectory, and it pulls the student off the trajectory it must reproduce at inference. The photograph is used for scoring only.

12.6 Evolving teacher

The frozen teacher never moves toward the high-score region; selection only filters what it produces. A self-distillation variant was therefore tested in which the teacher is an exponential moving average of the student. On every update the EMA teacher rolls out four fresh-seed candidates at $w = 1$, the DINOv2 patch scorer ranks them against the reference photograph on the fly, and the winner's EMA trajectory provides the targets. Runs start from the 118k averaged patch student (0.4865 / 22.05), use the 3k pool, 6,000 updates and one seed, and are compared with the same student continued against the frozen teacher (Table 37). Both the student and its EMA copy are evaluated.

Table 37: Evolving (EMA) teacher, seed 0, 6,000 updates from the 118k patch student. CompBench / GenEval2 of the raw student and of its EMA weights. $\bar{S}$ is the mean DINOv2 patch score of the teacher's fresh candidates at the end of training (0.60 at the start), H their normalised selection entropy at $T = 0.02$ (0.75 at the start).

Teacher	Selection	raw student	EMA weights	$\bar{S}$ at end	H at end
frozen (control)	DINOv2 patches, cached	0.4779 / 19.96	0.4847 / 20.68	–	–
EMA, decay 0.999	DINOv2 patches, online	0.1655 / 0.88	0.1789 / 2.09	0.03	0.93
EMA, decay 0.999	random	0.1546 / 1.00	0.1667 / 1.52	0.03	0.93
EMA, decay 0.999, frozen anchor 50%	DINOv2 patches, online	0.2095 / 6.28	0.2250 / 8.96	0.05	–
EMA, decay 0.9999	DINOv2 patches, online	0.4385 / 18.04	**0.4936 / 22.33**	0.62	0.75
EMA, decay 0.9999	random	0.4296 / 18.51	0.4917 / 22.03	0.61	–

A fast-tracking teacher (decay 0.999) collapses with or without selection and with a 50% frozen anchor: the candidates' score falls from 0.60 to 0.03 and their selection entropy rises from 0.75 to 0.93, so the scorer can no longer tell them apart, while the training loss falls and the student's samples turn into texture noise. The entropy of the online scores is an early warning of this collapse, visible thousands of updates before any image metric is computed. A slow teacher (decay 0.9999) is inert over 6,000 updates: the candidate score stays at 0.60 to 0.62, and selection within the slow loop is worth +0.002 CompBench and +0.3 GenEval2 against random ($p > 0.5$). Its EMA weights end 0.009 above the frozen-teacher control ($p = 0.01$), which is the effect of weight averaging, not of the moving teacher. Continuing the 118k student on the 3k pool with the frozen teacher lowers it (0.4865 to 0.4847), so the small pool is not a source of further gain either. The frozen teacher is kept.

12.7 Reference photograph aspect handling

Rescoring the 3k pool with the reference centre-cropped then resized, or resized then cropped, changes the top-1 pick on 12.2% and 11.8% of captions and lowers the recovered headroom from 43.6% to 41.0% and 40.9%. Cropping loses captioned objects at the edges, and mean-pooled patches tolerate the distortion of squashing. Squashing is therefore a design choice, not an oversight.

12.8 Patch mean against CLS token

The two DINOv2 readouts are the same model, the same decode, the same reference and the same normalisation; only the token summary differs. Offline, the patch mean recovers 43.6% of headroom against 39.9% for CLS. Trained, the patch arm is ahead by +0.0104 CompBench on identical prompts, positive on all three seeds, while the CLS arm's gain over random has an interval that includes zero. The CLS token is a global summary of what kind of image this is. The patch mean is a different summary of the same contextualised tokens; it may retain more of the object and layout information that compositional prompts test, but pooling removes

explicit patch correspondence and this ablation only shows that the readout matters, not which information carries the difference.

13 Qualitative examples

Figures 15 and 16 show prompts on which the scored student differs most from the random-selection student, chosen by the rule stated on each figure: the eight largest per-prompt margins, at most two per CompBench category, seed 0 of each arm, identical initial noise across all columns. They are cherry-picked by construction and show what the gain looks like where it is large, not typical behaviour. On CompBench the scored student wins 366 prompts, loses 229 and ties 1,803 within 0.1; on GenEval2 it wins 169, loses 140 and ties 491 of 800, and both students score below 0.05 on 27% of the prompts. The columns also show the base model at 4 steps with and without guidance, which fails, and the 28-step teacher, which the students approach. The recurring pattern in the wins is an object that the random-selection student drops or merges (the second object of a two-object prompt, the count of a numeracy prompt) and that the scored student renders, together with cleaner colour and material binding.

The exact reward. Figures 17 and 18 do the same for the exact-reward student of Section 11.7 against the argmax student, on the averaged seed-0 models of the 3k configuration, with the random-pick student as a third column. The rule is stated on each sheet: the eight largest per-prompt margins of the exact-reward student over argmax, at most two per CompBench category, followed by the four most negative margins, so the sheets show where the reward hurts as well as where it helps. Across all prompts of the two seed-0 models the exact-reward student wins 1,146 CompBench prompts and loses 892 (360 ties), and wins 421 GenEval2 prompts against 379, so the gain is broad rather than concentrated. Two things are visible down the columns. The exact-reward images are the most photographic of the three students: surfaces, materials and lighting are closer to the 28-step guided teacher and to a photograph, which is what the reward asks for and what the CMMD and precision numbers measure. And where the reward loses, the images are of comparable quality and the score flips are counts and placements at the evaluator's margin (four rabbits for three, a bird beside rather than on the balloon), not failures of the kind the base model shows at four steps.

Randomly selected prompts. The cherry-picked sheets show the extremes. Figures 19 and 20 show eight prompts per benchmark drawn uniformly at random with a fixed seed, no score involved, in the same layout. They are what typical behaviour looks like: the three students are close to one another on most prompts, the exact-reward student is slightly the most photographic in surfaces and lighting, and the per-prompt scores go both ways. On this draw the exact-reward student beats argmax on four of the eight CompBench prompts (mean 0.73 against 0.64) and on two of the eight GenEval2 prompts (mean 0.25 against 0.36); across all prompts of the same two models it wins 47.8% of CompBench prompts, loses 37.2% and ties the rest, and wins 52.6% of GenEval2 prompts. A win rate a few points above one half is what an average gain of 0.011 looks like prompt by prompt, and the sheets should be read that way: a broad small shift, not a uniform improvement.

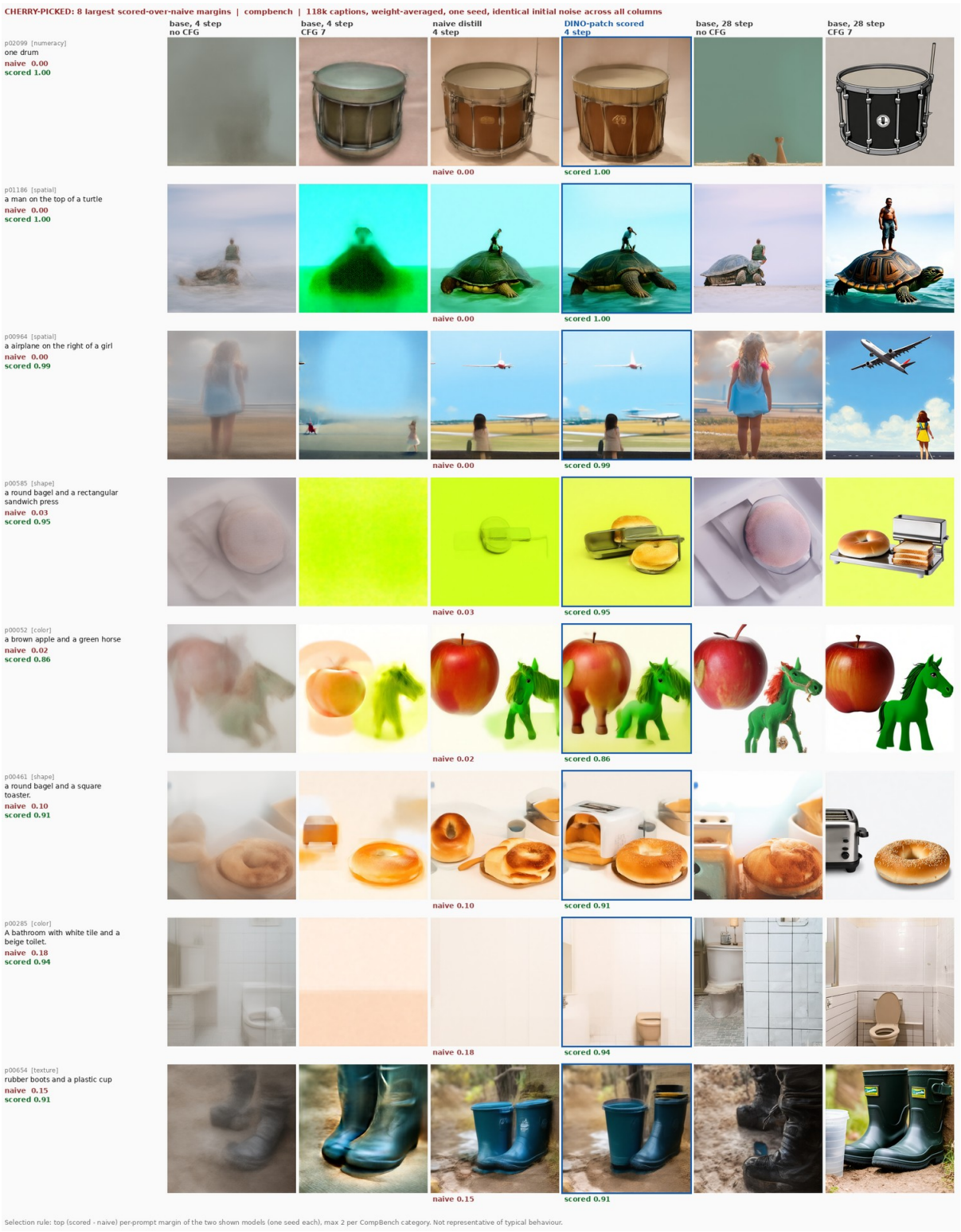

Figure 15: T2I-CompBench prompts with the largest scored-over-random margins at 118k. Columns: base model at 4 steps without and with guidance, random-selection student, DINOv2 patch student (framed), base model at 28 steps without and with guidance. Per-prompt scores under each student.

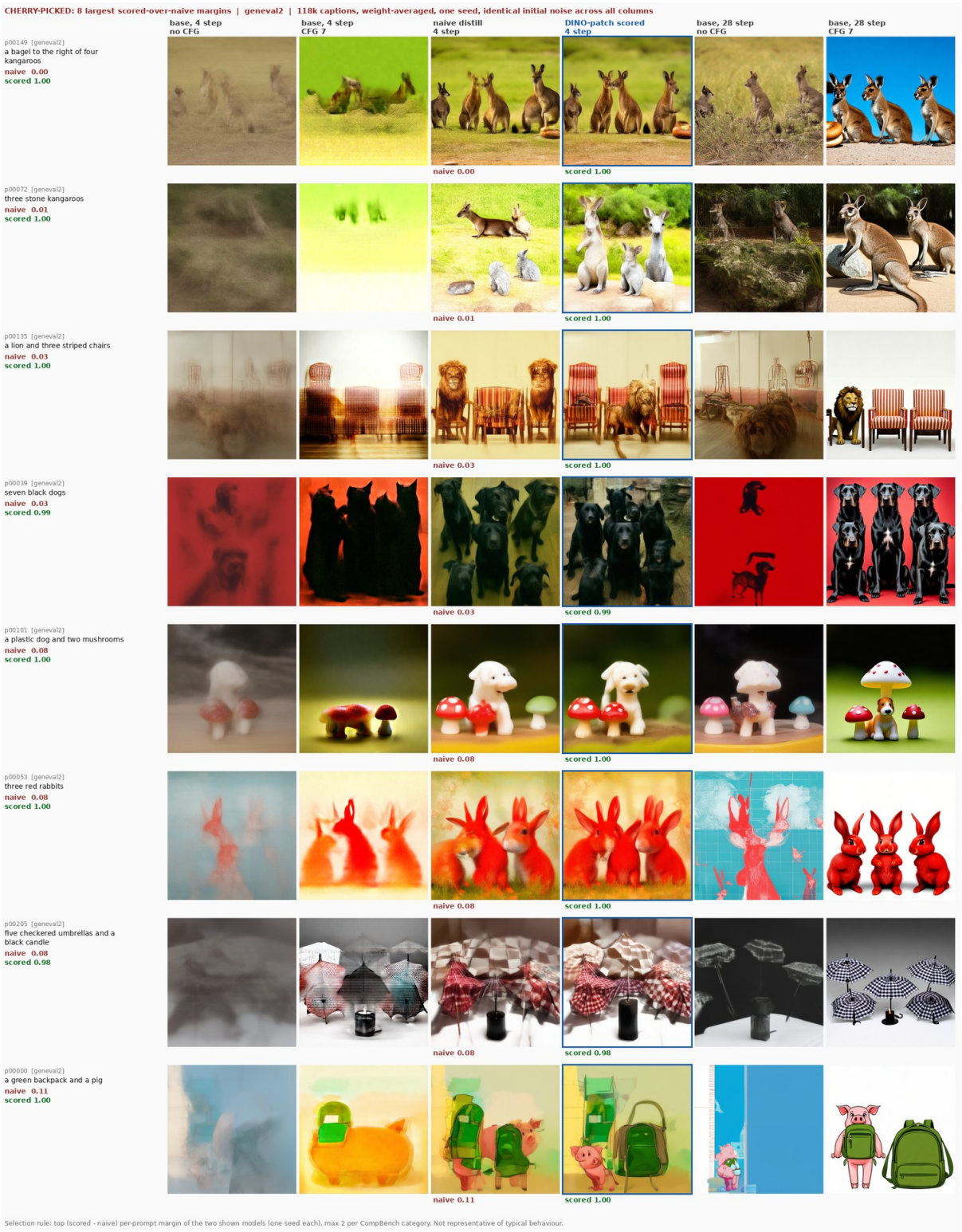

Figure 16: GenEval2 prompts with the largest scored-over-random margins at 118k, same layout as Figure 15.

CHERRY-PICKED: 8 largest (exact reward - argmax) margins, then 4 most negative | compbench | averaged checkpoints, seed 0, identical initial noise across all columns

	base, 4 step no CFG	base, 4 step CFG 7	random pick 4 step	argmax (DINO-patch) 4 step	argmax + exact reward 4 step	base, 28 step no CFG	base, 28 step CFG 7
p02099 [numeracy] one drum random 1.00 argmax 0.00 reward 1.00			random 1.00	argmax 0.00	reward 1.00		
p01177 [spatial] a person on the left of a vase random 0.00 argmax 0.00 reward 0.99			random 0.00	argmax 0.00	reward 0.99		
p00996 [spatial] a chair on the right of a giraffe random 0.93 argmax 0.00 reward 0.99			random 0.93	argmax 0.00	reward 0.99		
p00033 [color] a red banana and a green suitcase random 0.89 argmax 0.06 reward 0.95			random 0.89	argmax 0.06	reward 0.95		
p01293 [3d_spatial] a backpack hidden by a butterfly random 0.23 argmax 0.05 reward 0.84			random 0.23	argmax 0.05	reward 0.84		
p02184 [numeracy] two pillows and one microphone random 0.25 argmax 0.25 reward 1.00			random 0.25	argmax 0.25	reward 1.00		
p00163 [color] a gold backpack and a blue clock random 0.08 argmax 0.21 reward 0.95			random 0.08	argmax 0.21	reward 0.95		
p00754 [texture] a metallic keychain and wooden chopsticks random 0.45 argmax 0.11 reward 0.79			random 0.45	argmax 0.11	reward 0.79		

where the reward HURTS (most negative margins)

p01050 [spatial] a butterfly next to a train random 0.97 argmax 0.99 reward 0.00			random 0.97	argmax 0.99	reward 0.00		
p00926 [spatial] a bird on the top of a balloon random 0.99 argmax 0.99 reward 0.00			random 0.99	argmax 0.99	reward 0.00		
p00760 [texture] a metallic knife and a glass jar random 0.60 argmax 0.91 reward 0.15			random 0.60	argmax 0.91	reward 0.15		
p02242 [numeracy] two sinks and one helicopter random 0.75 argmax 1.00 reward 0.25			random 0.75	argmax 1.00	reward 0.25		

Selection rule: per-prompt (exact reward - argmax) benchmark margin of the two shown models (one seed each), max 2 per CompBench category, top 8 then bottom 4. Not representative of typical behaviour.

CHERRY-PICKED: 8 largest (exact reward - argmax) margins, then 4 most negative | geneval2 | averaged checkpoints, seed 0, identical initial noise across all columns

	base, 4 step no CFG	base, 4 step CFG 7	random pick 4 step	argmax (DINO-patch) 4 step	argmax + exact reward 4 step	base, 28 step no CFG	base, 28 step CFG 7
p00056 [geneval2] a monkey behind a penguin random 0.04 argmax 0.00 reward 1.00			random 0.04	argmax 0.00	reward 1.00		
p00006 [geneval2] four brown monkeys random 0.02 argmax 0.04 reward 1.00			random 0.02	argmax 0.04	reward 1.00		
p00029 [geneval2] four blue dogs random 0.89 argmax 0.01 reward 0.95			random 0.89	argmax 0.01	reward 0.95		
p00196 [geneval2] a glass toy and a spotted croissant random 0.02 argmax 0.03 reward 0.96			random 0.02	argmax 0.03	reward 0.96		
p00067 [geneval2] five glass monkeys random 0.92 argmax 0.11 reward 1.00			random 0.92	argmax 0.11	reward 1.00		
p00359 [geneval2] three yellow monkeys on top of a green clock random 0.22 argmax 0.13 reward 1.00			random 0.22	argmax 0.13	reward 1.00		
p00094 [geneval2] four red bears random 0.00 argmax 0.06 reward 0.92			random 0.00	argmax 0.06	reward 0.92		
p00267 [geneval2] a plastic car to the right of a plastic dog random 0.23 argmax 0.06 reward 0.88			random 0.23	argmax 0.06	reward 0.88		

where the reward HURTS (most negative margins)

p00053 [geneval2] three red rabbits random 1.00 argmax 0.99 reward 0.01			random 1.00	argmax 0.99	reward 0.01		
p00123 [geneval2] a cat under a donut, and a croissant random 0.16 argmax 1.00 reward 0.02			random 0.16	argmax 1.00	reward 0.02		
p00054 [geneval2] six black penguins random 0.09 argmax 0.95 reward 0.01			random 0.09	argmax 0.95	reward 0.01		
p00058 [geneval2] three metal mushrooms random 1.00 argmax 1.00 reward 0.09			random 1.00	argmax 1.00	reward 0.09		

Selection rule: per-prompt (exact reward - argmax) benchmark margin of the two shown models (one seed each), max 2 per CompBench category, top 8 then bottom 4. Not representative of typical behaviour.

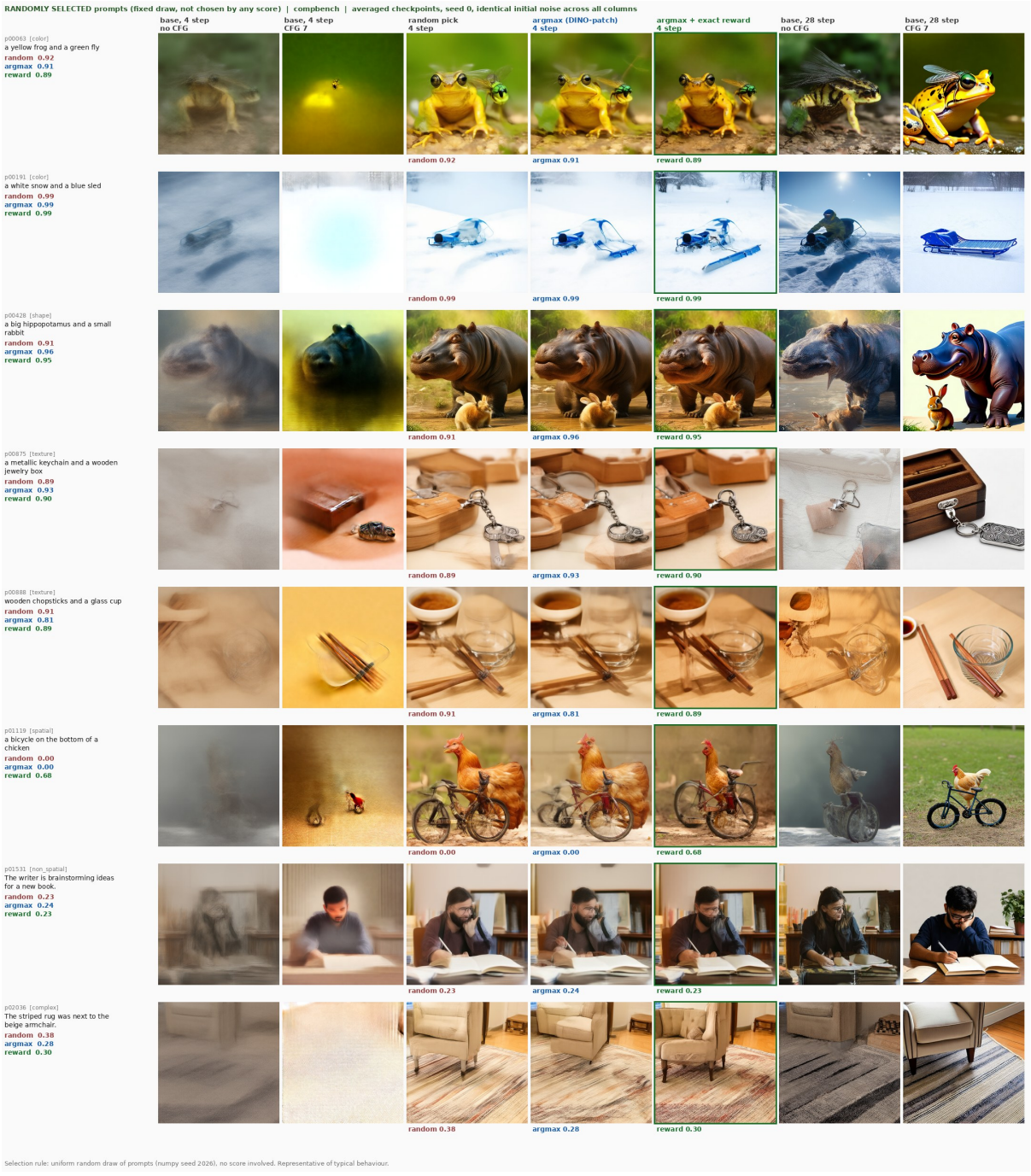

Figure 19: Eight T2I-CompBench prompts drawn uniformly at random (numpy seed 2026), no score involved; same columns and models as Figure 17.

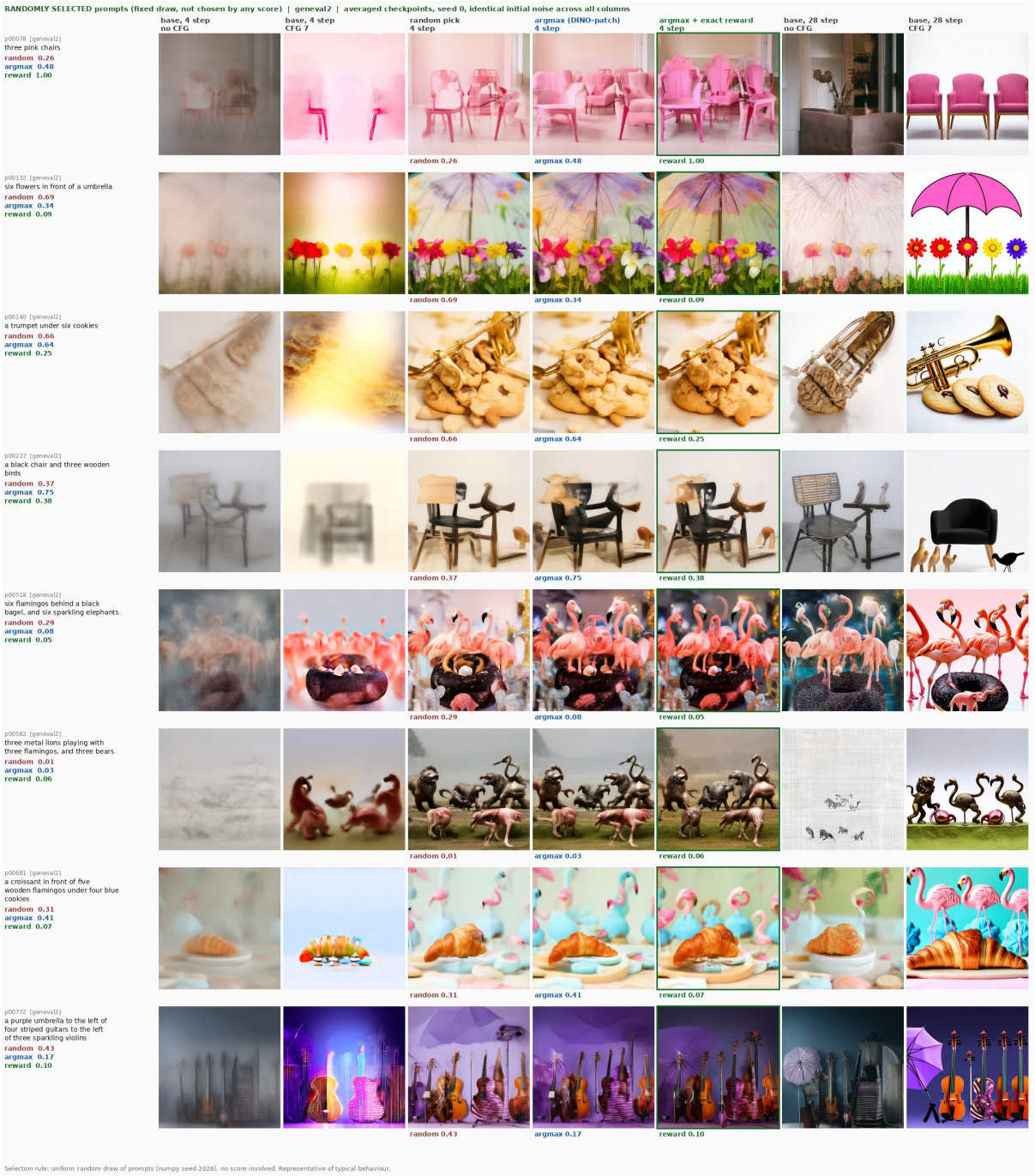

Figure 20: Eight GenEval2 prompts drawn uniformly at random, same rule and layout as Figure 19.

14 Cost

Per training caption the candidate pool costs 64 teacher forward passes (4 candidates, 8 steps, two guidance branches), four VAE decodes and five DINOv2 forward passes. Measured on one H100 at 512×512, generation takes 789 ms per caption, the four decodes 61 ms, DINOv2 scoring 32 ms, CLIP-I 48 ms and VQAScore 280 ms. End to end, DINOv2 selection costs 0.95 s per caption and VQAScore selection 1.19 s; candidate generation dominates and no scorer avoids it. Training costs 16 teacher passes and five student forward-backward passes per caption. A 3k run takes 1.6 hours on one H200; the 118k run takes 12.5 hours on four H200s. The DINOv2 scorer needs no text model and no large VLM at selection time, which is its practical advantage over VQAScore, ImageReward and PickScore. The exact Boltzmann arm of Section 11, which rolls out and trains on all four candidates, takes 2.6 s per update against 1.15 s for the single-candidate trainer (3.5 h against 1.2 h for a 3k run) for no detected gain over argmax. The latent scorer of Section 11.5 removes the decode and the DINO pass on the candidate (93 ms of the 950 ms per caption) at the price of a 24k-caption latent set (about 7 GPU-hours to generate) and a five-minute projector training.

15 Design choices

- **Fixed teacher targets, no EMA.** Every target is a function of the frozen teacher's trajectory. There is no bootstrapping and no target drift. Consistency models use an EMA or stop-gradient student copy; we do not.
- **Hard selection, stored once.** One candidate per caption, chosen offline. The trainer rolls out one trajectory per caption for every single-candidate arm, so their compute is identical; the exact-reward arm adds a decode and a DINOv2 pass per update (1.17 times the wall-clock). Exact Boltzmann weighting of all four candidates is not detectably better than argmax at about three times the wall-clock (211 against 69 minutes per 3k run), and the one-sample estimator of the same objective matches it after checkpoint averaging (Section 11). Four candidates are enough; eight add nothing (Section 10.6).
- **Guidance internalised.** The student trains with one conditional pass against guided teacher trajectories and is sampled at $w = 1$. This halves the student's inference cost relative to a guided sampler.
- **Re-roll instead of store.** Trajectories are re-rolled from integer seeds. The cache holds only seeds, scores and indices.
- **Reference photograph squashed to a square.** Cropping discards captioned content; see the aspect ablation.
- **Mean-pooled patches, CLS dropped.** The patch mean selects better than the CLS token (43.6% against 39.9% of the headroom offline, +0.010 CompBench trained). Pooling removes explicit patch correspondence; contextualised patch features may retain spatial information, but the ablation compares two readouts and does not establish that mechanism.
- **Weight averaging at scale.** The last five checkpoints are averaged because single checkpoints of a long run oscillate by more than the effect.
- **One sealed evaluation pool and fixed evaluation seeds.** Every arm and seed is scored on identical prompts and identical noise, so all contrasts are paired.

15.1 Positioning against related work

Three lines of work are adjacent to the reward result. Reward-guided latent consistency distillation [10] already combines consistency distillation with a differentiable reward on the student's clean predictions, studies latent proxy rewards, and documents over-optimisation of a direct RGB reward; distillation plus a differentiable reward is therefore not the novelty here. LaSRO [11] learns a latent-space surrogate reward for two-step diffusion models and adapts it online, so the projector result of Section 11.6 concerns one lightweight surrogate under one refresh regime and does not speak against latent surrogates in general. REPA [12] uses a pretrained visual encoder to improve generative training by aligning internal denoising representations to it, whereas here the encoder scores decoded predictions against a reference photograph. What is distinctive in this study is the supervision signal and how it is used: a reference photograph per caption, with no text-conditioned reward model, used twice, to select among the teacher's trajectories and to supervise the student's own predictions directly, and the measured gap between a latent scorer that is good enough to select (Section 11.5) and one that is not good enough to reward (Section 11.6). The exact reward did not show the over-optimisation reported for a direct RGB reward in [10]; whether that is the small weight, the restriction to the two least-noisy states, the short schedule or the hacking monitor is not established. The soft-weighting experiments of Section 11 justify a simple default and show that exact four-candidate weighting has not earned its cost; they do not establish equivalence between the rules, nor demonstrate any inherent failure of averaging over candidates.

16 Limitations

- **The scorer needs a reference photograph.** DINOv2 selection, and the latent scorer distilled from it, are only possible where each caption has a real image. They cannot be applied to synthetic prompt sets without one.
- **Grids do not nest.** The 4-step inference grid shares only its endpoints with the 8-step training grid, so the student is deployed at $\sigma = 0.858$ and 0.602, which it never saw as training inputs, and is never trained at $\sigma = 0.009$ (the last step is scaled by 0.009 and is immaterial). Grids with $K \in \{7, 10, 13, 16, \dots\}$ nest exactly; this was not used here.
- **Duplicate student inputs.** With five independent Δ_k draws, 81% of updates contain at least one repeated input state and the expected number of distinct inputs is 3.9 of 5. This does not change the objective, only its variance, and wastes about 22% of student passes.
- **Seeds.** Every headline number is three training seeds. The single-seed contrasts (the VQAScore arm at 118k, the batch-size row, the evolving-teacher runs) are marked as such and resolve about 0.01 CompBench.
- **Selection-rule contrasts are three-seed contrasts.** They resolve about 0.01 CompBench at the level of training runs. "No detected improvement" over argmax is the claim; equivalence is not.
- **Checkpoint averaging is part of the method at scale.** Without it the 118k gap is +0.007 instead of +0.0175. Averaging is applied identically to every arm, but the reported 118k numbers are for averaged weights.
- **Selection gain depends on candidate variance.** With a 28-step or a 16-step $w = 4.5$ teacher the within-caption score spread halves and the gain vanishes. Scored selection is a fixed-teacher gain, not a substitute for teacher quality.

- **VQAScore is circular.** Its gains share a model family with the BLIP-VQA categories of CompBench and the VLM judge of GenEval2. It is included as a reference, not as the proposed method.
- **The exact reward is a 3k-pool result.** Its size is the five-seed factorial's $+0.0055 \pm 0.0021$ CompBench over argmax with the caption order matched (Section 11.8); the first three runs, under a different order, read twice that. The fidelity gain is on every seed. Neither has been tested at 118k or with the stronger teachers. It also uses the reference photograph a second time, so it inherits the scorer's need for a real image per caption.
- **Metric properties.** GenEval2's geometric mean rewards partial credit, and CompBench 2D-spatial has dead prompts (Section 8.4). Both are constant across arms and do not reorder students, but they limit what absolute numbers mean.

17 Reproducibility

The release code is the `scored-consistency-distillation` branch of the project repository. It contains the pool builder, the candidate scorer, the trainer with the `random`, `dino_patch`, `boltzmann`, `boltzmann_sample`, `boltzmann_frozen`, `boltzmann_mc` and `uniform_visit` selectors, gradient accumulation, the gradient diagnostic, the latent scorer (latent generation, projector training, the `latent` selector), the reward arms (`--reward_mode proj` with its monitor and refresh, `--reward_mode rgb` for the exact reward, the reward-input builder, the held-out DINO monitor and the factorial launcher), the checkpoint averager, the evaluation scripts (CompBench with the one- and ten-image protocols, GenEval2, COCO VQAScore, FID, reference CMMD, precision and recall), the figure script and the LSF launch scripts. The release trainer was checked against the experimental trainers that produced the numbers in this document on all 909 parameter tensors after six optimizer updates under deterministic kernels. Every selector, gradient accumulation and both projector-reward arms are bit-identical, with original-versus-original controls at exactly zero. The exact-RGB reward arm is not bit-reproducible even against itself: the antialiased resize backward is a nondeterministic CUDA kernel and two runs of the original trainer differ by up to 7×10^{-7}; the release run differs from each original run by the same amount (7.2×10^{-7} and 7.5×10^{-7}), with identical printed losses at every update. That is agreement within the original's own nondeterminism, not bit identity, and these differences are far below anything that affects the training results. Every evaluation writes a pinned record with evaluator commits, package versions, pool hashes and the checkpoint hash, and fails hard on missing images. Training runs log per-state losses, selection statistics, throughput, sample grids and a source snapshot to Weights & Biases.

References

[1] P. Esser et al. Scaling rectified flow transformers for high-resolution image synthesis. ICML 2024.

[2] Y. Song, P. Dhariwal, M. Chen, I. Sutskever. Consistency models. ICML 2023.

[3] Y. Song, P. Dhariwal. Improved techniques for training consistency models. ICLR 2024.

[4] M. Oquab et al. DINOv2: learning robust visual features without supervision. TMLR 2024.

[5] Z. Lin et al. Evaluating text-to-visual generation with image-to-text generation. ECCV 2024.

[6] K. Huang et al. T2I-CompBench: a comprehensive benchmark for open-world compositional text-to-image generation. NeurIPS 2023.

[7] GenEval2: compositional text-to-image evaluation with atom-level VLM judging. 2025.

[8] S. Jayasumana et al. Rethinking FID: towards a better evaluation metric for image generation. CVPR 2024.

[9] J. Xu et al. ImageReward: learning and evaluating human preferences for text-to-image generation. NeurIPS 2023.

[10] J. Li, W. Feng, W. Chen, W. Y. Wang. Reward guided latent consistency distillation. TMLR 2024 (arXiv:2403.11027).

[11] Z. Jia, Y. Nan, H. Zhao, G. Liu. Reward fine-tuning two-step diffusion models via learning differentiable latent-space surrogate reward (LaSRO). arXiv:2411.15247, 2024.

[12] S. Yu, S. Kwak, H. Jang, J. Jeong, J. Huang, J. Shin, S. Xie. Representation alignment for generation: training diffusion transformers is easier than you think. ICLR 2025.

[13] Y. Kirstain et al. Pick-a-Pic: an open dataset of user preferences for text-to-image generation. NeurIPS 2023.